UD 2. Resumen. Fundamentos de programación para frontend con Javascript

2. Variables y tipos de datos
3. Estructuras de control
4. Funciones
5. Arrays
6. Objetos
7. JSON
8. Promesas
9. DOM y eventos
    - DOM
    - Eventos

2. Variables y tipos de datos

2.1 Sintaxis básica

2.1.1 Literales

Los literales son valores fijos que se escriben directamente en el código. No son variables, sino valores constantes que se asignan a variables o se utilizan directamente en las expresiones.

Ejemplos de Literales

- **Números**: Los números pueden ser enteros o decimales.

```
let num = 123; // Número entero
let num2 = 123.45; // Número decimal
```

- **Cadenas de texto (Strings)**: Las cadenas de texto pueden escribirse utilizando comillas dobles " o comillas simples '.

```
let cadena = "mi cadena"; // Usando comillas dobles
let cadena_comillas_simples = "mi cadena"; // Usando comillas simples
```

- **Booleanos**: Los valores booleanos son `true` o `false`.

```
const bandera = true; // Literal booleano verdadero
const cansado = false; // Literal booleano falso
```

- **Null**: El literal `null` representa la ausencia de un valor.

```
let objeto = null; // La variable 'objeto' no tiene ningún valor asignado
```

2.1.2 Identificadores en JavaScript

Los identificadores son nombres utilizados para identificar variables, funciones, clases u otros elementos dentro del código. Siguen ciertas reglas para ser válidos en JavaScript.

Reglas para crear identificadores

- Debe comenzar con una letra (a-z, A-Z), un guion bajo _ o un símbolo de dólar $.
- Después del primer carácter, pueden incluirse números (0-9), letras, guiones bajos _ y símbolos de dólar $.
- No pueden comenzar con un número.
- No pueden incluir caracteres especiales como +, -, *, &, etc.

Ejemplos válidos:

```
const nombre_variable_1 = "valor"; // Comienza con una letra
const _x = 10; // Comienza con un guion bajo
const $variable = "dólares"; // Comienza con un símbolo de dólar
```

Ejemplos no válidos:

```
const 1a = 20; // No válido: comienza con un número
const suma+ = 15; // No válido: contiene un carácter especial '+'
```

Ejemplos adicionales para mayor claridad

- **Variable válida y declaración**:

```
let resultado = 100; // 'resultado' es un identificador válido
```

Aquí, resultado es un identificador válido porque comienza con una letra y no contiene caracteres especiales.
Función con un identificador válido:

```
function calcularSuma(a, b) {
  return a + b;
}
```

calcularSuma es un identificador válido para una función que toma dos parámetros (a y b) y devuelve su suma. Identificadores no válidos y corrección:

```
// Incorrecto
// const 2variable = 10; // No puede comenzar con un número
// Correcto
const variable2 = 10; // Cambiado para comenzar con una letra
```

En este caso, 2variable es incorrecto porque comienza con un número. Lo corregimos cambiando el nombre a variable2, que comienza con una letra.

2.1.3 Palabras reservadas, Unicode y punto y coma

Palabras reservadas

Las palabras reservadas son identificadores que tienen un significado especial en el lenguaje y, por lo tanto, no pueden ser utilizados como nombres de variables, funciones o etiquetas. Estas palabras están reservadas por el lenguaje para mantener la sintaxis y las reglas de JavaScript.

Ejemplos:

Algunas de las palabras reservadas más comunes en JavaScript son:

- `if`, `else`, `for`, `while`, `switch`: utilizadas para el control de flujo.
- `var`, `let`, `const`: utilizadas para declarar variables.
- `function`: utilizada para declarar funciones.
- `return`: utilizada para devolver un valor de una función.
- `class`, `extends`, `super`: utilizadas en la programación orientada a objetos con clases.
- **`try`, `catch`, `finally` **: utilizadas para el manejo de excepciones.

Ejemplo en Código:

```
// Esto es correcto
let nombre = "Juan";
// Esto es incorrecto y causará un error
let if = "algo"; // "if" es una palabra reservada y no puede usarse como nombre de
variable
// Correcto uso de una palabra reservada
if (nombre === "Juan") {
console.log("El nombre es Juan");
}
```

Unicode en JavaScript

JavaScript utiliza Unicode, un estándar de codificación que permite representar la mayoría de los caracteres escritos del mundo. Esto incluye caracteres de alfabetos no latinos, símbolos especiales, emojis, etc.

Ejemplo:

Puedes utilizar caracteres Unicode directamente en tus cadenas de texto o variables. Incluso, puedes utilizar Unicode en los nombres de las variables, aunque esto no es recomendable.

Ejemplo en Código:

```
// Ejemplo con caracteres Unicode en una cadena
let saludo = "Hola, Τζάβασκριπτ"; // Javascript en griego
// Ejemplo con caracteres Unicode en nombres de variables (poco común pero
posible)
let π = 3.14159;
console.log(saludo); // Output: Hola, Τζάβασκριπτ
console.log("Valor de pi: " + π); // Output: 3.14159
```

Escape Unicode: JavaScript también permite usar secuencias de escape Unicode para representar caracteres. Estas secuencias comienzan con \u seguido de un código hexadecimal de cuatro dígitos.

```
let corazon = "\u2764";
console.log(corazon); // ❤
```

Uso del Punto y Coma en JavaScript

El punto y coma (😉 en JavaScript se utiliza para terminar una instrucción. Sin embargo, JavaScript tiene una característica llamada Automatic Semicolon Insertion (Inserción Automática de Punto y Coma, ASI), que permite al intérprete agregar puntos y comas automáticamente en algunos casos donde faltan.

Ejemplos:

```
// Uso explícito del punto y coma
let nombre = "Juan";
console.log(nombre);
// JavaScript puede agregar un punto y coma automáticamente
let apellido = "Pérez"
console.log(apellido); // Aunque falta el punto y coma, no causará un error
```

Importancia del Punto y Coma: Aunque ASI (Automatic Semicolon Insertion) ayuda a evitar errores, hay casos donde no poner el punto y coma puede llevar a un comportamiento inesperado. Por ejemplo:

```
// Ejemplo donde la falta de punto y coma causa un problema
let suma = 5 + 5
(function() {
  console.log("Esto es una función IIFE");
})();
// Esto causará un error, porque JavaScript intenta interpretar como una sola
```

```
instrucción:
// let suma = 5 + 5(function() { console.log("Esto es una función IIFE");})();
```

2.2 Comentarios

Los comentarios en JavaScript funcionan de manera similar a los de Java. Son útiles para agregar notas, explicaciones o descripciones dentro del código, que no son ejecutadas por el intérprete. Los comentarios ayudan a que el código sea más legible y comprensible tanto para el autor como para otros desarrolladores que lo lean en el futuro.

2.2.1 Comentario de una línea

Para comentar una sola línea en JavaScript, se utiliza //. Todo lo que siga a // en esa línea será ignorado por el intérprete.

Ejemplo:

```
// Esto es un comentario de una sola línea
let x = 5; // También se puede hacer aquí
```

En este caso, la primera línea es un comentario completo y la segunda línea tiene un comentario al final, explicando que se está asignando el valor 5 a la variable x.

2.2.2 Comentario de múltiples líneas

Para comentar varias líneas a la vez, se utiliza /* */. *Todo el texto entre* / y */ será ignorado por el intérprete.

Ejemplo:

```
/*
Este es un comentario
de múltiples líneas
que puede abarcar varias
líneas de código
*/
let y = 10;
```

Este tipo de comentario es útil cuando necesitas explicar un bloque de código o dejar una nota más extensa.

2.3 Variables

2.3.1 Variables con var

`var` es la forma tradicional de declarar variables en JavaScript, y ha sido utilizada desde las primeras versiones del lenguaje. Las variables declaradas con `var` tienen un **ámbito de función**, lo que significa que su visibilidad se limita a la función en la que se declara. Sin embargo, si se declara una variable con `var` fuera de cualquier

función, ésta tendrá un ámbito global. Esto quiere decir que la variable es accesible en todo el documento. Ejemplo básico de `var`

```
var a = 10;
if (a > 9) {
  var b = 2;
}
console.log(a); // Output: 10
console.log(b); // Output: 2 (b existe fuera del bloque)
```

En este ejemplo, la variable b sigue existiendo y es accesible fuera del bloque `if`. Esto se debe a que `var` no tiene un ámbito de bloque, sino de función o global. Este comportamiento puede llevar a errores si no se tiene en cuenta.

```
function mifuncion() {
  var c = 3;
  console.log(b); // Output: 2 (b sigue existiendo dentro de la función)
  console.log(c); // Output: 3
}
mifuncion();
console.log(c); // Error: c no está definida fuera de la función
```

Aquí, la variable c se declara dentro de la función mifuncion, por lo que no es accesible fuera de ella. En otros lenguajes de programación como Java, la variable b no existiría fuera del bloque {} donde fue declarada. Sin embargo, en JavaScript, al usar `var`, la variable b tiene visibilidad fuera del bloque donde se definió.

> Evita usar `var` para declarar variables. Usa `let` o `const` en su lugar.

2.3.2 Hoisting

El **hoisting** es un comportamiento en JavaScript en el que las declaraciones de variables y funciones se mueven al comienzo del ámbito donde están declaradas. Este comportamiento afecta únicamente a las declaraciones, no a las asignaciones.

```
function mifuncion() {
  console.log(c); // Output: undefined
  var c = 3;
  console.log(c); // Output: 3
}
```

En este ejemplo, aunque la declaración de la variable c aparece después del primer `console.log`, JavaScript mueve la declaración al inicio de la función. Sin embargo, la asignación de c a 3 no se mueve, por lo que inicialmente c tiene el valor `undefined`. Es importante comprender que `undefined` significa que la variable está declarada aunque aún no tiene un valor definido. En lenguajes como Java, el código anterior generaría un error en el primer `console.log`, ya que la variable c no existiría aún. Puedes experimentar con el hoisting

utilizando la sentencia `debugger` para detener la ejecución del código y observar el comportamiento de las variables en el navegador.

```
function pruebaHoisting() {
  debugger;

  console.log(x);
  var x = 10;
  console.log(x);
}
pruebaHoisting();
```

2.3.3 Variables con let

La palabra reservada `let` se introdujo en ES6 y permite declarar variables con un ámbito de bloque. Esto significa que la variable sólo es accesible dentro del bloque {} donde se declaró.

```
let a = 1;
if (a > 0) {
  let b = 2;
  console.log(b); // Output: 2
}
console.log(b); // Error: b no está definida
```

2.3.4 Declaraciones con const

La palabra reservada `const` también fue introducido en ES6 y se utiliza para declarar constantes, es decir, variables cuyo valor no puede ser reasignado después de su declaración. Sin embargo, en el caso de objetos, no se puede modificar la referencia, pero sí el contenido.

```
const mensaje = "hola mundo";
const persona = { nombre: "Lucia", apellidos: "Molina" };
// mensaje = "adios"; // Error: no se puede reasignar una constante
persona.nombre = "Antonio"; // Esto está permitido
// persona = {}; // Error: no se puede reasignar una constante
```

El ámbito de `let` y `const` es de bloque, lo que significa que sólo existen dentro del bloque {} donde fueron declarados, al igual que en otros lenguajes como Java.

```
let x = 10;
{
  let y = 20;
  const pi = 3.14;
  console.log(x); // Output: 10
  console.log(y); // Output: 20
```

```
  console.log(pi); // Output: 3.14
}
console.log(y); // Error: y no está definida
console.log(pi); // Error: pi no está definida
```

> Opta en primer lugar por usar `const` para declarar todas las variables. Si el valor de la variable necesita cambiar, entonces usa `let`.

2.3.5 Tipado dinámico

A diferencia de lenguajes como Java, donde el tipo de una variable se declara de manera explícita y no puede cambiar, en JavaScript el tipo de una variable puede cambiar durante la ejecución. Esto se conoce como tipado dinámico.

```
let variable = 1;
console.log(typeof variable); // Output: "number"
variable = "Ahora soy un string";
console.log(typeof variable); // Output: "string"
```

> Aunque JavaScript permite el **tipado dinámico**, es preferible evitar cambiar el tipo de una variable una vez que se ha establecido, ya que esto puede llevar a errores difíciles de depurar.

Para depurar el código en JavaScript, utiliza herramientas como el `debugger` del navegador para observar cómo se comportan las variables y comprender mejor el flujo de tu programa.

2.4 Operadores

2.4.1 Operadores aritméticos

Los operadores aritméticos realizan cálculos con valores numéricos. `+` también concatena cadenas.

```
const a = 10;
const b = 3;
console.log(a + b); // 13: suma
console.log(a - b); // 7: resta
console.log(a * b); // 30: multiplicación
console.log(a / b); // 3.333...: división
console.log(a % b); // 1: resto
console.log(a ** b); // 1000: potencia
console.log("Hola " + "mundo"); // "Hola mundo": concatenación
```

Los operadores unarios `+` y `-` indican signo. `++` y `--` incrementan o decrementan una variable en uno; en forma prefija devuelven el valor nuevo y en forma postfija el valor anterior.

```
let contador = 2;
console.log(++contador); // 3
```

```
console.log(contador++); // 3; contador pasa a valer 4
console.log(-contador); // -4
```

2.4.2 Asignación

`=` asigna el valor de la derecha a la variable de la izquierda. Los operadores compuestos actualizan la variable aplicando una operación.

```
let total = 10;
total += 5; // Equivale a total = total + 5
total -= 2; // Equivale a total = total - 2
total *= 3; // Equivale a total = total * 3
total /= 2; // Equivale a total = total / 2
total %= 4; // Equivale a total = total % 4
total **= 2; // Equivale a total = total ** 2
```

Operadores de comparación

Comparan dos valores y devuelven un booleano. Para comparar igualdad, se recomienda `===` y `!==`, que no convierten los tipos automáticamente.

```
const edad = 20;
console.log(edad === 20); // true
console.log(edad !== 18); // true
console.log(edad > 18); // true
console.log(edad < 18); // false
console.log(edad >= 20); // true
console.log(edad <= 21); // true
console.log(5 == "5"); // true: convierte el tipo
console.log(5 === "5"); // false: compara valor y tipo
```

Operadores lógicos

`&&` (AND) requiere que ambas condiciones sean verdaderas; `||` (OR) requiere que una lo sea; `!` (NOT) invierte el valor. `??` devuelve el valor de la derecha solo cuando el de la izquierda es `null` o `undefined`.

```
const tieneEntrada = true;
const esMayor = false;
console.log(tieneEntrada && esMayor); // false
console.log(tieneEntrada || esMayor); // true
console.log(!tieneEntrada); // false
const nombre = null;
console.log(nombre ?? "Invitado"); // "Invitado"
```

`&&` y `||` aplican cortocircuito: no evalúan la expresión de la derecha si el resultado ya queda determinado por la izquierda.

Operador condicional

El operador ternario `condición ? valorSiVerdadero : valorSiFalso` elige entre dos expresiones.

```
const edadMinima = 18;
const mensaje = edadMinima >= 18 ? "Acceso permitido" : "Acceso denegado";
```

Operadores bit a bit

Operan sobre la representación binaria de números enteros. Los más comunes son AND `&`, OR `|`, XOR `^`, NOT `~`, desplazamiento a la izquierda `<<` y desplazamientos a la derecha `>>` y `>>>`.

```
console.log(5 & 3); // 1
console.log(5 | 3); // 7
console.log(5 ^ 3); // 6
console.log(5 << 1); // 10
```

Operadores de tipo y pertenencia

`typeof` devuelve el tipo de un valor, `instanceof` comprueba si un objeto deriva de un constructor y `in` comprueba si una propiedad existe en un objeto.

```
const persona = { nombre: "Ana" };
console.log(typeof persona.nombre); // "string"
console.log(persona instanceof Object); // true
console.log("nombre" in persona); // true
```

Precedencia y paréntesis

Al combinar operadores, JavaScript aplica reglas de precedencia. Los paréntesis hacen explícito el orden deseado.

```
const resultado = 2 + 3 * 4; // 14: la multiplicación se evalúa primero
const agrupado = (2 + 3) * 4; // 20
```

2.5 Tipos de datos

2.5.1 Tipos primitivos

Los tipos de datos primitivos en JavaScript son los más básicos y no pueden ser divididos en partes más pequeñas. Los tipos de datos primitivos en JavaScript son inmutables, lo que significa que una vez que un valor primitivo se ha creado, no se puede cambiar o modificar. Sin embargo, la variable que contiene ese valor primitivo puede ser reasignada para contener un valor diferente. Por ejemplo, si tienes una variable que contiene un número y luego le asignas un nuevo número, lo que realmente sucede es que la variable ahora apunta a un nuevo valor, pero el valor original permanece inalterado.

```
let x = 10;
x = 20; // La variable 'x' ahora referencia al valor 20, pero el valor 10 no ha
cambiado,
```

simplemente ya no está referenciado por 'x'. Por ejemplo, si intentas modificar una cadena de texto (string), se generará una nueva cadena en lugar de alterar la original.

```
let saludo = "Hola";
let saludoModificado = saludo.toUpperCase(); // 'saludoModificado' es una nueva
cadena "HOLA"
console.log(saludo); // "Hola" - La cadena original no ha cambiado
```

2.5.2 Tipo Number

Numbers

El tipo number representa tanto números enteros como de punto flotante. JavaScript no distingue entre tipos de números como lo hacen otros lenguajes de programación.

```
const entero = 42;
const decimal = 3.14;
console.log(entero); // Output: 42
console.log(decimal); // Output: 3.14
const decimalPreciso = 1.00000000000000000000001; // Pérdida de precisión
const hexadecimal = 0xff; // Valor Hexadecimal
const binario = 0b0101; // Valor Binario
console.log(binario); // Output: 5
console.log(hexadecimal); // Output: 255
console.log(decimalPreciso); // Output: 1
```

Métodos de Numbers

Los números disponen de métodos para convertirlos o dar formato a sus valores.

```
const numero = 123.456;
console.log(numero.toFixed(2)); // "123.46"
console.log(Number.isInteger(numero)); // false
```

```
console.log(Number.parseInt("42")); // 42
console.log(Number.parseFloat("3.14")); // 3.14
```

2.5.3 Tipo String

Strings

El tipo `string` representa texto. Se puede escribir con comillas simples, dobles o con backticks, que permiten crear plantillas de cadena.

```
const cadena1 = "Hola, Mundo";
const cadena2 = 'Hola, Mundo';
const cadena3 = `Hola, Mundo`;
```

Métodos String

Aunque los tipos primitivos en JavaScript no poseen métodos ni propiedades inherentes, en la práctica, se comportan como si los tuvieran. Esto es posible porque JavaScript realiza un proceso automático conocido como auto-boxing. Cuando intentas acceder a un método o propiedad de un valor primitivo, JavaScript temporalmente convierte ese valor primitivo en un objeto envoltorio (wrapper object) correspondiente. JavaScript tiene clases nativas que actúan como envoltorios para cada tipo primitivo, como String, Number, Boolean, Symbol, y BigInt. Estos objetos envoltorios permiten que los primitivos "hereden" métodos y propiedades útiles, como `toUpperCase()` para cadenas o `toFixed()` para números.

```
let texto = "Hola";
console.log(texto.toUpperCase()); // "HOLA"
```

En el ejemplo anterior, cuando se llama al método `toUpperCase()` en la cadena de texto texto, JavaScript convierte temporalmente el valor primitivo "Hola" en un objeto String. Este objeto permite el uso del método `toUpperCase()`. Después de que el método se ejecuta, el objeto temporal se descarta y el resultado es devuelto como un nuevo valor primitivo.

Métodos de Strings

Los strings pueden contener caracteres especiales que necesitan ser "escapados" utilizando la barra invertida (\). Algunos de los más comunes son:

```
const str = "It's a beautiful day"; // Escapando la comilla simple
const str2 = 'She said "Hello"'; // Escapando la comilla doble
const str3 = "Una línea\nOtra línea"; // Nueva línea
const str4 = "C:\\Users\\Usuario"; // Barra invertida
```

Concatenación de Strings

La concatenación de strings se puede hacer utilizando el operador + o el método concat().

```
const saludo = "Hola";
const nombre = "Juan";
const mensaje = saludo + " " + nombre + "!";
console.log(mensaje); // "Hola Juan!"
console.log(saludo.concat(" ", nombre, ".")); // "Hola Juan."
```

Comparación de Strings

Los strings en JavaScript se comparan utilizando operadores como ==, ===, !=, !==, <, >, etc. Las comparaciones de strings son sensibles a mayúsculas y minúsculas y se realizan en base al valor Unicode de los caracteres.

```
const str1 = "abc";
const str2 = "def";
const str3 = "ABC";
console.log(str1 < str2); // true, porque "a" es menor que "d" en Unicode
console.log(str1 < str3); // true, porque "abc" es lexicográficamente menor que
"ABC"
```

Conversión de Otros Tipos a String

Puedes convertir otros tipos de datos a strings usando el método String() o toString().

```
let num = 123;
let str = String(num); // "123"
let str2 = num.toString(); // "123"
```

Las cadenas en JavaScript vienen con varios métodos y propiedades útiles que permiten manipular y analizar el texto de manera eficiente.

- Propiedad `length` La propiedad `length` devuelve el número de caracteres en una cadena, incluidos los espacios.

  ```
  let texto = "Hola, Mundo!";
  console.log(texto.length); // 12
  ```

Métodos de Manipulación:

- **`toUpperCase()` y `toLowerCase()`**: Convertir la cadena a mayúsculas o minúsculas.

```
let texto = "Hola, Mundo!";
console.log(texto.toUpperCase()); // "HOLA, MUNDO!"
console.log(texto.toLowerCase()); // "hola, mundo!"
```

- `charAt(index)`: Obtener el carácter en una posición específica.

```
let texto = "Hola";
console.log(texto.charAt(1)); // "o"
```

- `substring(start, end)`: Extraer una subcadena entre dos índices (el índice de end no se incluye).

```
let texto = "JavaScript";
console.log(texto.substring(0, 4)); // "Java"
```

- `slice(start, end)`: Similar a substring(), pero permite índices negativos para contar desde el final de la cadena.

```
let texto = "JavaScript";
console.log(texto.slice(-6)); // "Script"
```

- `split(separator)`: Divide la cadena en un array de subcadenas, utilizando un separador especificado.

```
let texto = "Hola, Mundo!";
let palabras = texto.split(" ");
console.log(palabras); // ["Hola,", "Mundo!"]
```

- `trim()`: Elimina los espacios en blanco al principio y al final de la cadena.

```
let texto = " Hola, Mundo! ";
console.log(texto.trim()); // "Hola, Mundo!"
```

- `replace(searchValue, newValue)`: Reemplaza una parte de la cadena con otra.

```
let texto = "Hola, Mundo!";
let nuevoTexto = texto.replace("Mundo", "JavaScript");
console.log(nuevoTexto); // "Hola, JavaScript!"
```

String templates

Los Template Strings (o Template Literals) son una característica avanzada de JavaScript introducida en ECMAScript 2015 (ES6) que ofrece una forma más flexible y potente de trabajar con cadenas de texto. A continuación, se explican en profundidad sus características, usos, y beneficios. Los Template Strings se crean usando backticks (``) en lugar de comillas simples (' ') o dobles (" "). Esto permite a los desarrolladores crear cadenas de texto de manera más dinámica y legible.

```
let saludo = `Hola Mundo`;
```

- **Interpolación de Expresiones** Una de las características más poderosas de los Template Strings es la capacidad de interpolar (insertar) expresiones JavaScript directamente dentro de la cadena usando ${}.

  ```
  let nombre = "Juan";
  let edad = 30;
  let mensaje = `Mi nombre es ${nombre} y tengo ${edad} años.`;
  console.log(mensaje); // "Mi nombre es Juan y tengo 30 años."
  ```

Dentro de ${}, puedes colocar cualquier expresión válida de JavaScript, como operaciones matemáticas, llamadas a funciones, o incluso expresiones ternarias.

```
let a = 5;
let b = 10;
let resultado = `La suma de ${a} y ${b} es ${a + b}.`;
console.log(resultado); // "La suma de 5 y 10 es 15."
```

- Multi-línea Los Template Strings permiten la creación de cadenas de texto que abarcan múltiples líneas sin necesidad de concatenar strings o usar secuencias de escape como \n.

  ```
  const mensaje = `Este es un mensaje
  que se extiende
  a través de varias líneas.`;
  console.log(mensaje);
  ```

Este uso de Template Strings mejora enormemente la legibilidad y la gestión de textos largos o estructuras HTML en JavaScript.

- **Tags o Funciones de Plantilla** Los template strings también soportan una característica avanzada llamada tagged templates o funciones de plantilla. Permiten que una función procese un template string antes de que se interprete. Esta funcionalidad es útil para crear soluciones avanzadas como plantillas personalizadas, traducciones, o escapado de HTML.

  ```
  function etiqueta(strings, ...valores) {
    console.log(strings); // ["Hola ", " soy ", ""]
  ```

```
  console.log(valores); // ["Juan", 30]
  return `Saludos, ${valores[0]}. Tienes ${valores[1]} años.`;
}
let nombre = "Juan";
let edad = 30;
let resultado = etiqueta`Hola ${nombre} soy ${edad}`;
console.log(resultado); // "Saludos, Juan. Tienes 30 años."
```

En este ejemplo, strings es un array que contiene las partes literales de la cadena, mientras que valores es un array que contiene las expresiones interpoladas.

> Es importante recordar las funciones de plantilla de las template strings ya que las usaremos en los styled components de React.

- **Raw Strings** Los template strings también tienen un método incorporado llamado String.raw que permite obtener la representación "cruda" de la cadena, es decir, sin procesar las secuencias de escape.

```
let path = String.raw`C:\Development\profile\aboutme.html`;
console.log(path); // "C:\Development\profile\aboutme.html"
```

En una cadena normal, tenemos que escapar las barras invertidas:

```
let path = "C:\\Development\\profile\\aboutme.html";
console.log(path); // "C:\Development\profile\aboutme.html"
```

En el primer caso, las secuencias de escape no se procesan y se mantienen tal cual.

- **Tagged Templates para Sanitizar Entradas** Un uso práctico de los tagged templates es para sanitizar entradas, como escapar caracteres potencialmente peligrosos para evitar ataques de inyección de HTML o SQL. Aquí un ejemplo simplificado de cómo podrías usarlo:

```
function escapeHTML(strings, ...values) {
  let resultado = strings[0];
  for (let i = 0; i < values.length; i++) {
    const escaped = String(values[i])
      .replace(/&/g, "&amp;")
      .replace(/</g, "&lt;")
      .replace(/>/g, "&gt;")
      .replace(/"/g, "&quot;")
      .replace(/'/g, "&#39;");
    resultado += escaped + strings[i + 1];
  }
  return resultado;
}

const userInput = "<script>alert('Malicious Code');</script>";
const sanitizedString = escapeHTML`Usuario dijo: ${userInput}`;
```

```
console.log(sanitizedString);
// "Usuario dijo: &lt;script&gt;alert(&#39;Malicious
Code&#39;);&lt;/script&gt;"
```

En este ejemplo, escapeHTML es una función que toma una cadena de plantilla y reemplaza caracteres especiales para prevenir ataques de inyección de HTML.

- Usos Prácticos Comunes de las template strings Construcción de HTML dinámico:

```
const items = ["Manzana", "Banana", "Cereza"];
const listaHTML = `<ul>${items.map((item) => `<li>${item}</li>`).join("")}
</ul>`;
console.log(listaHTML);
```

Generación de consultas SQL dinámicas:

```
let tableName = "users";
let columns = ["name", "age", "email"];
let sqlQuery = `SELECT ${columns.join(", ")} FROM ${tableName}`;
console.log(sqlQuery); // "SELECT name, age, email FROM users"
```

Búsqueda en String

- `includes(substring)`: Devuelve `true` si la cadena contiene la subcadena especificada.

```
let texto = "Hola, Mundo!";
console.log(texto.includes("Mundo")); // true
```

- `indexOf(substring) y lastIndexOf(substring)`: Devuelve la posición de la primera o última aparición de la subcadena.

```
let texto = "Hola, Mundo! Hola!";
console.log(texto.indexOf("Hola")); // 0
console.log(texto.lastIndexOf("Hola")); // 13
```

2.5.4 Tipo Boolean

En JavaScript, un valor booleano es un tipo de dato que solo puede tener uno de dos valores posibles: `true` (verdadero) o `false` (falso). Este tipo de dato es fundamental para realizar comparaciones y controlar el flujo del programa mediante estructuras condicionales como `if`, `else`, `while`, y `for`.

```
const esVerdadero = true;
const esFalso = false;
```

En JavaScript, cualquier valor puede ser convertido a un booleano utilizando la función Boolean(), o simplemente evaluándolo en un contexto que requiere un valor booleano (como en una condición `if`).

Conversiones de Booleanos

Valores false

Los siguientes valores se convierten a `false` cuando se evalúan en un contexto booleano:

- `false`
- 0 (el número cero)
- "" (cadena de texto vacía)
- `null`
- `undefined`
- NaN (Not a Number)

```
console.log(Boolean(0)); // false
console.log(Boolean("")); // false
console.log(Boolean(null)); // false
console.log(Boolean(undefined)); // false
console.log(Boolean(NaN)); // false
```

Valores true

Cualquier valor que no sea uno de los "falsy" mencionados anteriormente es considerado "truthy", es decir, se convierte a `true` en un contexto booleano.

```
console.log(Boolean(1)); // true
console.log(Boolean("Hola")); // true
console.log(Boolean([])); // true (un array vacío)
console.log(Boolean({})); // true (un objeto vacío)
console.log(Boolean(function () {})); // true (una función)
```

Igualdad y coerción

En JavaScript, existen dos operadores de igualdad principales: el operador de igualdad == y el operador de igualdad estricta ===.

- **Igualdad simple (==)**:

El operador `==` compara valores después de convertirlos a un tipo común, lo que puede producir resultados inesperados.

```
console.log(1 == "1"); // true
console.log(true == 1); // true
console.log(null == undefined); // true
```

- **Igualdad estricta (===)**: compara valor y tipo sin conversión.

```
console.log(1 === "1"); // false
console.log(true === 1); // false
console.log(null === undefined); // false
console.log(1 === 1); // true
```

> Usar el operador de igualdad estricta === es recomendable porque evita errores sutiles que pueden surgir por la coerción (conversión) de tipos cuando se usa ==. Con ===, te aseguras de que no haya conversión de tipos automática, lo que hace que el código sea más predecible y menos propenso a errores.

2.5.5 Otros tipos primitivos

Hay seis tipos de datos primitivos en ES6+:

Undefined

El tipo `undefined` representa una variable que ha sido declarada pero no inicializada. Cuando una variable es declarada sin asignarle un valor, su tipo es `undefined`.

```
let variable;
console.log(variable); // Output: undefined
```

Null

El tipo `null` es un valor especial que representa la ausencia intencional de cualquier valor u objeto. Es un valor asignable y se utiliza comúnmente para inicializar variables que se espera que luego contengan un objeto.

```
const obj = null;
console.log(obj); // Output: null
```

2.5.6 Tipos de datos de objeto

Objetos (Object)

Un objeto es una colección de propiedades y métodos. Cada propiedad es una asociación entre una clave y un valor. Los objetos se crean utilizando llaves {} o utilizando constructores como Object.

```
let persona = {
nombre: 'Juan',
  edad: 30,
};
console.log(persona.nombre); // Output: Juan
console.log(persona.edad); // Output: 30
```

Arrays (Array)

Un array es una colección ordenada de elementos, que puede contener valores de cualquier tipo. Los arrays se crean utilizando corchetes [] o utilizando el constructor Array.

```
let colores = ["rojo", "verde", "azul"];
console.log(colores[0]); // Output: rojo
console.log(colores.length); // Output: 3
```

Funciones (Function)

Una función es un bloque de código que se puede definir y ejecutar cuando se necesite. Las funciones en JavaScript también son un tipo especial de objeto.

```
function saludar(nombre) {
  return `Hola, ${nombre}`;
}
console.log(saludar("Carlos")); // Output: Hola, Carlos
```

Como son objetos, las funciones pueden ser pasadas como argumentos a otras funciones, devueltas por otras funciones y asignadas a variables.

```
function saludar(nombre) {
  return `Hola, ${nombre}`;
}
let saludo = saludar;
console.log(saludo("Carlos")); // Output: Hola, Carlos
```

Mapas (Map)

Un mapa es una nueva estructura de datos introducida en ES6 que permite almacenar pares clavevalor, donde cualquier tipo de datos puede ser una clave.

```
let mapa = new Map();
mapa.set("nombre", "Pedro");
mapa.set(1, "uno");
console.log(mapa.get("nombre")); // Output: Pedro
console.log(mapa.get(1)); // Output: uno
```

Sets (Set)

Un set es una colección de valores únicos, es decir, no permite duplicados. Los sets se utilizan cuando se necesita asegurarse de que una colección de valores no tenga duplicados.

```
let set = new Set([1, 2, 3, 3, 4]);
console.log(set); // Output: Set(4) {1, 2, 3, 4}
```

Además de los tipos de datos mencionados, JavaScript tiene otros tipos y construcciones especiales.

2.5.7 Tipos de datos especiales

BigInt

Introducido en ES2020, el tipo BigInt permite representar números enteros más grandes que el máximo permitido por el tipo number.

```
let numeroGrande = BigInt(9007199254740991);
console.log(numeroGrande); // Output: 9007199254740991n
```

TypedArray

Los TypedArray son objetos similares a los arrays que proporcionan una vista sobre buffers binarios. Se utilizan cuando se trabaja con datos binarios en JavaScript.

```
let buffer = new ArrayBuffer(16);
let vista = new Uint8Array(buffer);
console.log(vista.length); // 16
```

2.5.8 Coerción

La coerción en JavaScript es el proceso mediante el cual el lenguaje convierte automáticamente un valor de un tipo de dato a otro, cuando es necesario para que una operación tenga sentido o sea posible. Esta característica puede facilitar la escritura de código, pero también puede llevar a comportamientos inesperados si no se comprende bien cómo funciona. Existen dos tipos de coerción en JavaScript:

1. Coerción implícita Ocurre cuando JavaScript convierte automáticamente un valor de un tipo a otro sin que el programador lo indique explícitamente.

    - **Coerción a string (concatenación)**:

```
let resultado = "5" + 3; // "53"
```

Aquí, el número 3 se convierte en una cadena '3' para que la operación de concatenación pueda ocurrir.

- **Coerción a número (suma, resta, etc.)**:

```
let resultado = "5" - 3; // 2
```

En este caso, la cadena '5' se convierte en el número 5 para que la operación de resta pueda realizarse.

- **Coerción a booleano**:

```
if ("") {
  console.log("Esto no se muestra");
}
```

Aquí, la cadena vacía '' se convierte en `false`, por lo que el bloque `if` no se ejecuta. 2. Coerción explícita Ocurre cuando el programador convierte manualmente un valor de un tipo a otro utilizando funciones o constructores específicos.

- **Convertir a string**:

```
let numero = 123;
let texto = String(numero); // "123"
```

- **Convertir a número**:

```
let texto = "456";
let numero = Number(texto); // 456
let n = parseInt("123"); // 123
let f = parseFloat("1234.12"); // 1234.12
```

- **Convertir a booleano**:

```
let valor = 1;
let booleano = Boolean(valor); // true
```

- Ejemplos de coerción en operaciones comunes
  - **Igualdad flexible (==)**: convierte los operandos antes de compararlos.

    ```
    console.log(5 == "5"); // true
    ```

  - **Igualdad estricta (===)**: compara sin convertir los operandos.

    ```
    console.log(5 === "5"); // false
    ```

  - **Operaciones matemáticas**: los operadores convierten a número cuando es necesario.

    ```
    console.log("10" * 2); // 20
    ```

Peligros y Consideraciones

- **Inconsistencia**: La coerción implícita puede llevar a resultados inesperados, especialmente cuando se usan operadores como == en lugar de ===.
- **Legibilidad del Código**: La coerción explícita es generalmente preferible porque hace el código más claro y predecible, ya que es evidente cuándo y cómo se realiza la conversión de tipos.

Ejemplos problemáticos

- **Concatenación de strings con números** y **comparaciones flexibles**:

```
console.log("10" + 1); // "101" (concatenación, no suma)
console.log([] == false); // true
```

2.6 Dates

2.6.1 Date formats

Las fechas se pueden expresar como cadenas ISO 8601. Para mostrarlas según el idioma y la región, se puede usar `Intl.DateTimeFormat`.

```
const fecha = new Date("2024-05-20T09:30:00Z");
console.log(new Intl.DateTimeFormat("es-ES", { dateStyle: "long"
```

```
}).format(fecha));
```

2.6.2 Métodos Date

El constructor `Date` crea fechas y horas. `Date.now()` devuelve los milisegundos transcurridos desde el 1 de enero de 1970. `getMonth()` devuelve un mes entre 0 y 11.

```
const ahora = new Date();
console.log(ahora.getFullYear());
console.log(ahora.getMonth() + 1);
console.log(ahora.getDate());
console.log(Date.now());
```

2.7 Math

`Math` es un objeto integrado que proporciona propiedades y métodos para operaciones matemáticas, como redondeos, potencias y trigonometría. No se instancia; sus propiedades y métodos se consultan directamente.

```
const resultado = Math.PI * Math.pow(2, 3); // Potencia
console.log(Math.abs(-5)); // Valor absoluto 5
console.log(Math.round(Math.PI)); // Redondea: 3
console.log(Math.ceil(1.9)); // Techo: 2
console.log(Math.floor(1.6)); // Piso: 1
console.log(Math.max(1, 3, 8, 9, 123, 5, 0)); // Máximo: 123
console.log(Math.min(5, -2, 100, 10, 9)); // Mínimo: -2
console.log(Math.sqrt(9)); // Raíz cuadrada: 3
console.log(Math.sin(Math.PI / 2)); // Función seno: 1
```

> El objeto Math es nativo de JavaScript y no necesita ser importado.

2.7.1 Random

```
console.log(Math.random()); // Número aleatorio entre 0 y 1
```

2.8 Pedir datos al usuario

`prompt()` muestra un cuadro de diálogo y devuelve el texto introducido por el usuario.

```
const nombre = prompt("¿Cómo te llamas?");
```

2.9 Mostrar mensajes de alerta

`alert()` muestra un mensaje en un cuadro de diálogo del navegador.

```
alert("Hola, mundo");
```

2.10 Desestructuración

La desestructuración en JavaScript es una característica del lenguaje introducida en ECMAScript 6 (ES6) que permite extraer valores de arrays o propiedades de objetos y asignarlos a variables de manera más concisa y clara. Esto es particularmente útil para trabajar con estructuras complejas de datos, como objetos anidados o arrays multidimensionales. A continuación, se explica la desestructuración en detalle para ambos casos: arrays y objetos.

2.10.1 Desestructuración de Arrays

La desestructuración de arrays permite extraer valores de un array y asignarlos a variables de manera directa. La sintaxis básica es:

```
const array = [1, 2, 3, 4];
// Desestructuración
const [a, b, c] = array;
console.log(a); // 1
console.log(b); // 2
console.log(c); // 3
```

Características clave:

- **Asignación por posición**: Los valores son asignados a las variables en función de su posición en el array. En el ejemplo anterior, a toma el valor del primer elemento del array, b del segundo, y así sucesivamente.
- **Valores por defecto**: Puedes asignar valores por defecto a las variables en caso de que los elementos del array sean `undefined`.

  ```
  const array = [1, 2];
  const [a, b, c = 3] = array;
  console.log(a); // 1
  console.log(b); // 2
  console.log(c); // 3
  ```

- **Omisión de valores**: Si no te interesa algún valor intermedio del array, puedes omitirlo usando una coma.

  ```
  const array = [1, 2, 3, 4];
  const [a, , c] = array;
  ```

```
console.log(a); // 1
console.log(c); // 3
```

2.10.2 Desestructuración de Objetos

La desestructuración de objetos permite extraer propiedades de un objeto y asignarlas a variables. La sintaxis básica es:

```
const objeto = { x: 1, y: 2, z: 3 };
// Desestructuración
const { x, y } = objeto;
console.log(x); // 1
console.log(y); // 2
```

Características clave:

- **Asignación por nombre**: A diferencia de los arrays, la desestructuración de objetos se basa en los nombres de las propiedades. En el ejemplo anterior, x y y son asignados a las variables correspondientes con los mismos nombres.
- **Asignación a nuevos nombres de variables**: Puedes asignar las propiedades a variables con nombres diferentes.

  ```
  const objeto = { x: 1, y: 2 };
  const { x: a, y: b } = objeto;
  console.log(a); // 1
  console.log(b); // 2
  ```

- **Valores por defecto**: Similar a los arrays, puedes definir valores por defecto para propiedades que no existen o son `undefined`.

  ```
  const objeto = { x: 1 };
  const { x, y = 2 } = objeto;
  console.log(x); // 1
  console.log(y); // 2
  ```

- **Desestructuración anidada**: Puedes desestructurar objetos anidados.

  ```
  const objeto = {
    a: 1,
    b: {
      c: 2,
      d: 3,
    },
  ```

```
};
const { b: { c, d } } = objeto;
console.log(c); // 2
console.log(d); // 3
```

2.10.3 Aplicaciones Prácticas

La desestructuración es muy útil en diversas situaciones, como:

- **Intercambio de valores**: Intercambiar valores entre dos variables sin una variable temporal.

```
let a = 1,
  b = 2;
[a, b] = [b, a];
console.log(a); // 2
console.log(b); // 1
```

- **Extracción de datos de funciones**: Cuando una función retorna un objeto, puedes desestructurarlo directamente en la llamada.

```
function obtenerCoordenadas() {
  return { x: 10, y: 22 };
}
const { x, y } = obtenerCoordenadas();
console.log(x, y); // 10 22
```

- **Parámetros de función**: Puedes desestructurar directamente en los parámetros de una función.

```
function imprimirCoordenadas({ x, y }) {
  console.log(`Coordenadas: ${x}, ${y}`);
}
imprimirCoordenadas({ x: 10, y: 22 });
```

3. Estructuras de control {#estructuras-de-control}

3.1 Estructuras condicionales: `if`, `switch`, ternario

A continuación, se repasa el uso de las estructuras `if`, `else`, `switch`, y otros conceptos relacionados.

3.1.1 Estructura `if`, `else if`, `else`

```
const hora = 10;
if (hora < 12) {
```

```
  console.log("Buenos días");
} else if (hora < 18) {
  console.log("Buenas tardes");
} else {
  console.log("Buenas noches");
}
// Salida: Buenos días
```

La declaración `else if` permite agregar múltiples condiciones entre un `if` inicial y un `else` final.

```
const hora = 18;
if (hora < 12) {
  console.log("Buenos días");
} else if (hora < 18) {
  console.log("Buenas tardes");
} else {
  console.log("Buenas noches");
}
// Salida: Buenas noches
```

```
const nota = 8;
if (nota < 5) {
  console.log("Suspenso");
} else if (nota < 6) {
  console.log("Aprobado");
} else {
  console.log("Excelente");
}
```

3.1.2 Evaluación implícita de valores a booleanos

JavaScript convierte automáticamente los valores en condiciones a booleanos (`true` o `false`). Esto significa que no solo los valores `true` y `false` son válidos, sino también otros tipos de datos, como números, cadenas, arrays, objetos, etc.

```
const bandera = true;
if (bandera) {
  console.log("Entra en el if");
}
```

Los valores que se convierten en `false` (valores *falsy*) son `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined` y `NaN`. Cualquier otro valor es *truthy*; por ejemplo, las cadenas no vacías y todos los objetos, incluidos los arrays y objetos vacíos.

```
if ([]) {
  console.log("Un array vacío es truthy");
}

if ("") {
  console.log("Este mensaje no se muestra");
}
```

3.1.3 Variables sin inicializar y valores falsy

Una variable declarada con `let` sin valor asignado contiene `undefined`. Una condición como `if (variable)` no comprueba si se asignó un valor: comprueba si el valor es *truthy*. Por ejemplo, `0` y `""` son valores asignados, pero se consideran *falsy*. Una declaración `const` siempre debe incluir un valor inicial.

```
let variable;
if (variable === undefined) {
  console.log("No tiene un valor definido");
} else {
  console.log("Tiene un valor definido");
}
```

Esta comparación tampoco permite distinguir entre una variable que no recibió asignación y otra a la que se asignó explícitamente `undefined`.

3.1.4 Uso de operadores lógicos

Operador lógico AND (&&)

El operador `&&` evalúa de izquierda a derecha y se detiene al encontrar un valor *falsy*. Devuelve ese valor; si todos son *truthy*, devuelve el último. Este comportamiento se conoce como evaluación de corto circuito.

```
function A() {
  console.log("called A");
  return null;
}
function B() {
  console.log("called B");
  return true;
}
console.log(A() && B());
```

En este ejemplo, B nunca es llamada porque A() retorna `null`, que es un valor falsy. Como resultado, la evaluación se detiene y se retorna `null`.

Operador lógico OR (||)

El operador `||` evalúa de izquierda a derecha y se detiene al encontrar un valor *truthy*. Devuelve ese valor; si todos son *falsy*, devuelve el último.

```
function A() {
  console.log("called A");
  return [];
}
function B() {
  console.log("called B");
  return true;
}
console.log(A() || B());
```

Aquí, B no es llamada porque A() retorna un array `[]`, que es un valor *truthy*. La evaluación se corta y se devuelve ese array. El operador `!` invierte el valor booleano de una expresión y puede usarse para negar una condición.

```
const tienePermiso = false;
if (!tienePermiso) {
  console.log("Acceso denegado");
}
```

3.1.5 Estructura `switch`

La estructura `switch` se utiliza para seleccionar uno entre varios bloques de código para ejecutar, según el valor de una expresión.

```
function aNotaNumerica(calificacion) {
  let nota = 0;
  switch (calificacion) {
    case "Suspenso":
      nota = 1;
      break;
    case "Aprobado":
      nota = 5;
      break;
    case "Sobresaliente":
      nota = 9;
      break;
    default:
      nota = 0;
  }
  return nota;
}

let calificacion = "Sobresaliente";
console.log(aNotaNumerica(calificacion)); // 9
```

```
calificacion = "Otra cosa";
console.log(aNotaNumerica(calificacion)); // 0
```

`switch` compara la expresión con cada `case` usando igualdad estricta (`===`) y ejecuta el primer bloque coincidente. `break` termina el `switch`; si se omite, la ejecución continúa en el siguiente `case` (*fall-through*). `default` se ejecuta cuando ningún caso coincide y es opcional.

3.1.6 Operador ternario

El operador ternario (? 😃 es una forma concisa de escribir un `if-else`. Se utiliza para evaluar una expresión y retornar un valor basado en la condición. Es una expresión a diferencia de `if` que es una estructura de control.

```
const activado = true;
const color = activado ? "green" : "red";
console.log(color); // "green"
const edad = 18;
const bebida = edad >= 18 ? "cerveza" : "cocacola";
console.log(bebida); // "cerveza"
// Gestionar valores nulos
let person = { name: "Luis" };
console.log(person ? person.name : ""); // "Luis"
person = null;
console.log(person ? person.name : ""); // ""
```

Este operador es útil para realizar asignaciones rápidas basadas en una condición, manteniendo el código limpio y legible.

3.2 Bucles

Los bucles repiten un bloque de código. `for` suele ser práctico cuando se controla un contador; `while` comprueba la condición antes de cada repetición y puede no ejecutarse ninguna vez; `do...while` la comprueba después y se ejecuta al menos una vez.

3.2.1 `for`

El bucle `for` reúne en su cabecera la inicialización, la condición y la actualización del contador. Es práctico cuando se conoce el número de repeticiones o se necesita controlar un índice.

```
for (let i = 0; i < 5; i++) {
  console.log("Iteración número: " + i);
}
```

3.2.2 `while`

El bucle `while` se utiliza cuando las repeticiones dependen de una condición. Esta se comprueba antes de cada vuelta, por lo que el cuerpo puede no ejecutarse si inicialmente es falsa.

```
let i = 0;
while (i < 5) {
  console.log("Iteración número: " + i);
  i++;
}
```

3.2.3 do...`while`

El bucle do...`while` es similar a `while`, pero con una diferencia importante: el bloque de código se ejecuta al menos una vez antes de que la condición sea evaluada.

```
let i = 0;
do {
  console.log("Iteración número: " + i);
  i++;
} while (i < 5);
```

3.2.4 `for`...in

El bucle `for...in` recorre las claves de las propiedades enumerables de un objeto, incluidas las heredadas. Úsalo principalmente con objetos, no con arrays: recorre claves y no valores, y puede incluir propiedades que no sean índices.

```
const persona = { nombre: "Juan", edad: 30, ciudad: "Madrid" };
for (const clave in persona) {
  console.log(clave + ": " + persona[clave]);
}
```

Si solo necesitas las propiedades propias del objeto, puedes recorrer `Object.keys(persona)` con `for...of`.

3.2.5 `for`...of

El bucle `for...of` recorre los valores de un objeto iterable, como un array, una cadena, un `Map` o un `Set`.

```
const array = ["a", "b", "c"];
for (const letra of array) {
  console.log(letra);
}
```

3.2.6 `forEach`

`forEach` ejecuta una función una vez por cada elemento del array. La función que recibe se llama *callback* y puede tener nombre; no tiene que ser anónima.

```
const numeros = [1, 2, 3, 4, 5];

function mostrarNumero(numero) {
  console.log(numero);
}

numeros.forEach(mostrarNumero);
```

También puedes escribir la función directamente dentro de `forEach`, por ejemplo como función flecha:

```
numeros.forEach((numero) => console.log(numero));
```

`forEach` no permite detener el recorrido con `break` ni saltar una vuelta con `continue`. Para controlar el bucle, usa `for`, `while` o `for...of`.

3.2.7 Sentencias `break` y `continue`

`break` termina el bucle más cercano. `continue` omite el resto de la iteración actual y pasa a la siguiente. `break` también termina un `switch`.

```
for (let numero = 0; numero < 10; numero++) {
  if (numero === 2) {
    continue;
  }
  if (numero === 7) {
    break;
  }
  console.log(numero);
}
// Muestra 0, 1, 3, 4, 5 y 6
```

3.3 Manejo de excepciones: `throw`, `try`, `catch` y `finally`

`throw` lanza una excepción. El bloque `try` contiene el código que puede fallar, `catch` recibe y gestiona el error, y `finally` se ejecuta tanto si hubo una excepción como si no.

```
function dividir(dividendo, divisor) {
  if (divisor === 0) {
    throw new Error("No se puede dividir entre cero");
  }
  return dividendo / divisor;
}

try {
  console.log(dividir(10, 0));
```

```
} catch (error) {
  console.error(error.message);
} finally {
  console.log("Fin del cálculo");
}
```

4. Funciones

4.1 Declaración y uso de funciones

Una **función** es un bloque de código con nombre (o asignado a una variable) que realiza una tarea. Puede recibir datos de entrada mediante **parámetros** y devolver un resultado con `return`. Así se puede reutilizar una operación sin repetir su implementación. Una llamada a una función ejecuta su cuerpo; el valor que devuelve se puede guardar, mostrar o utilizar en otra operación. Conviene que cada función tenga una responsabilidad clara y un nombre que describa lo que hace.

```
function sumar(a, b) {
  return a + b;
}

const resultado = sumar(349, 123);
console.log(resultado); // 472
console.log(sumar(10, 5) * 2); // 30
```

En este ejemplo, `a` y `b` son parámetros; `349` y `123` son los argumentos que se pasan al llamar a la función. `return` entrega el resultado al código que la llamó. Si una función no ejecuta `return`, el resultado de la llamada es `undefined`. No hay que confundir devolver un valor con mostrarlo: `console.log` escribe un mensaje en la consola, mientras que `return` proporciona un valor que otra parte del programa puede reutilizar.

4.1.1 Declaraciones de función y hoisting

La forma habitual de definir una función es mediante una **declaración de función**. JavaScript permite llamarla antes de la línea donde aparece su declaración: esta característica se conoce como *hoisting*.

```
console.log(multiplicar(5, 7)); // 35

function multiplicar(a, b) {
  return a * b;
}
```

El hoisting no significa que el código se reordene literalmente: describe cómo JavaScript prepara las declaraciones al crear el ámbito. Esta posibilidad puede dificultar la lectura; por claridad, suele ser preferible definir la función antes de usarla. Esta regla se aplica a las declaraciones completas de función, no a cualquier

variable que contenga una función. Por ejemplo, intentar llamar a una función asignada a `const` antes de su definición provoca un error porque esa variable todavía no está inicializada.

4.1.2 Funciones como expresiones

Las funciones también son valores. Una **expresión de función** permite asignar una función a una variable, guardarla en una estructura o pasarla como argumento. Si no tiene nombre, se denomina **función anónima**.

```
const cuadrado = function (x) {
  return x * x;
};
console.log(cuadrado(8)); // 64
```

Una función asignada a `const` no se puede llamar antes de inicializar esa variable. A diferencia de una declaración de función, la expresión no está disponible mediante hoisting.

Las funciones también se pueden pasar como argumentos. En ese caso se pasa la función sin paréntesis, para entregar la función en sí y no ejecutar su resultado en ese momento. Una función que recibe otra función como argumento se denomina **función de orden superior**. Así se puede separar la operación común (mostrar el área) de la regla concreta que calcula cada figura:

```
function areaRectangulo(base, altura) {
  return base * altura;
}
function areaTriangulo(base, altura) {
  return (base * altura) / 2;
}
function mostrarArea(figura, base, altura, calcularArea) {
  console.log(`El área del ${figura} es: ${calcularArea(base, altura)}`);
}

mostrarArea("rectángulo", 5, 10, areaRectangulo); // 50
mostrarArea("triángulo", 5, 10, areaTriangulo); // 25
```

La función `mostrarArea` no necesita conocer la fórmula: recibe el cálculo que debe realizar. A la función que se pasa para que otra la ejecute se la suele llamar **callback**. Este patrón permite reutilizar una operación con comportamientos diferentes.

4.1.3 Recursividad

Una función es **recursiva** cuando se llama a sí misma. Debe tener un caso base que detenga las llamadas; si no, seguirá ejecutándose hasta producir un error por exceso de llamadas.

```
function factorial(n) {
  if (n <= 1) return 1; // Caso base
  return n * factorial(n - 1);
}
```

```
console.log(factorial(5)); // 120
```

En cada llamada, `n` disminuye en uno hasta llegar al caso base. El caso base evita que la función se llame indefinidamente; el paso recursivo debe acercar los datos a ese caso. Esta solución asume un entero positivo. Si los valores proceden del usuario, hay que validarlos antes de llamar a la función. La recursividad resulta útil para ciertos problemas, aunque para repeticiones sencillas un bucle suele ser más fácil de seguir y no acumula llamadas en la pila.

4.2 Funciones flecha

Las **funciones flecha**, introducidas en ES6, ofrecen una sintaxis concisa. Si el cuerpo contiene una sola expresión, su resultado se devuelve automáticamente. Con varias instrucciones se usan llaves y `return` explícito. Por tanto, estas dos funciones hacen lo mismo:

```
const sumar = (a, b) => a + b;
const sumarConReturn = (a, b) => {
  return a + b;
};

console.log(sumar(4, 7)); // 11
console.log(sumarConReturn(4, 7)); // 11
```

Con un único parámetro se pueden omitir los paréntesis; con cero o más de uno, son necesarios. Cuando una flecha devuelve un objeto literal, hay que rodearlo con paréntesis para que las llaves no se interpreten como el cuerpo de la función:

```
const duplicar = numero => numero * 2;
const crearPunto = (x, y) => ({ x: x, y: y });
console.log(duplicar(6)); // 12
console.log(crearPunto(3, 4)); // { x: 3, y: 4 }
```

Una diferencia importante es que las funciones flecha **no tienen un `this` propio**: usan el valor del contexto donde se crearon. En cambio, un método tradicional llamado desde un objeto puede utilizar `this` para acceder a las propiedades de ese objeto. Por eso, no conviene definir como flecha un método que necesite su propio `this`:

```
const calculadora = {
  valor: 10,
  sumar(n) {
    return this.valor + n;
  },
};

console.log(calculadora.sumar(5)); // 15
```

4.3 Parámetros y argumentos

Los **parámetros** son los nombres que aparecen en la definición de la función; los **argumentos** son los valores concretos que se entregan al llamarla. Se asignan en orden, de izquierda a derecha. Si se pasan menos argumentos que parámetros, los que falten valen `undefined`, salvo que tengan un valor predeterminado.

4.3.1 Parámetros opcionales y valores por defecto

Se puede asignar un valor predeterminado a un parámetro. JavaScript lo utiliza cuando el argumento se omite o vale `undefined` (no cuando vale `null`). Los parámetros con valores predeterminados se colocan normalmente después de los obligatorios para que la llamada sea más fácil de entender.

```
function presentar(nombre, prefijo = "Hola") {
  return `${prefijo}, ${nombre}`;
}

console.log(presentar("Ana")); // "Hola, Ana"
console.log(presentar("Ana", "Bienvenida")); // "Bienvenida, Ana"
```

4.3.2 Parámetro rest

El parámetro **rest** (`...`) permite que una función reciba un **número variable de argumentos**. Todos los valores sobrantes se recogen en un **array real**.

Reglas:

- Se escribe con tres puntos antes del nombre: `...numeros`.
- Debe ser el **último** parámetro de la función.
- Solo puede haber **uno** por función.
- Si no se pasan argumentos adicionales, el array queda vacío (`[]`).

En el ejemplo se recorren esos valores con un bucle y se devuelve el total:

```
function sumarTodos(...numeros) {
  let total = 0;
  for (const numero of numeros) {
    total += numero;
  }
```

```
  return total;
}

console.log(sumarTodos(1, 2, 3, 4)); // 10
console.log(sumarTodos()); // 0
```

Paso a paso con `sumarTodos(1, 2, 3, 4)`:

1. `numeros` pasa a ser `[1, 2, 3, 4]`.
2. `total` empieza en `0`.
3. El bucle acumula cada elemento: `0+1 → 1`, `1+2 → 3`, `3+3 → 6`, `6+4 → 10`.
4. Se devuelve `10`.

Con `sumarTodos()` el array está vacío, el bucle no se ejecuta y se devuelve `0`.

El parámetro rest puede combinarse con parámetros normales, que van siempre primero:

```
function saludar(saludo, ...nombres) {
  return nombres.map((nombre) => `${saludo}, ${nombre}`);
}

console.log(saludar("Hola", "Ana", "Luis")); // ["Hola, Ana", "Hola, Luis"]
```

> **No confundir con spread:** ambos usan `...`, pero hacen lo contrario. *Rest* (en la definición) **agrupa** valores en un array; *spread* (en la llamada) **expande** un array en valores sueltos, por ejemplo `sumarTodos(...[1, 2, 3])`.

4.3.3 Objeto `arguments`

En las funciones tradicionales existe el objeto `arguments`, que contiene los argumentos recibidos. No es un array real: permite consultar su longitud y acceder a cada posición, pero no ofrece directamente los métodos de los arrays. Tampoco está disponible dentro de las funciones flecha. En código moderno se suele preferir el parámetro rest, que sí crea un array y deja explícito qué parámetros son fijos y cuáles son variables.

```
function mostrarArgumentos() {
  for (let i = 0; i < arguments.length; i++) {
    console.log(arguments[i]);
  }
}

mostrarArgumentos("rojo", "verde", "azul");
```

4.4 Spread y desestructuración

Spread y rest usan los mismos tres puntos (`...`), pero cumplen papeles distintos según dónde aparezcan: **rest reúne** varios argumentos en un array al definir una función; **spread expande** los elementos de un array al hacer una llamada.

4.4.1 Operador spread

El operador **spread** (`...`) hace lo contrario que rest: toma un array (o cualquier iterable) y **lo expande en valores sueltos**, como si los hubieras escrito uno a uno separados por comas.

Es útil cuando una función espera argumentos individuales pero los datos están en un array. Por ejemplo, `Math.max` y `Math.min` no aceptan un array: esperan `Math.max(1, 2, 3)`.

```
const numeros = [1, 2, 3, 4, 5, 6];

console.log(Math.max(numeros)); // NaN: recibe UN único argumento (el array)
console.log(Math.max(...numeros)); // 6: equivale a Math.max(1, 2, 3, 4, 5, 6)
console.log(Math.min(...numeros)); // 1
```

Spread también sirve para **copiar y combinar arrays** sin modificar los originales:

```
const a = [1, 2];
const b = [3, 4];

const unidos = [...a, ...b, 5]; // [1, 2, 3, 4, 5]
const copia = [...a]; // nuevo array con los mismos elementos
```

Diferencia entre rest y spread

Los dos usan `...`; lo que los distingue es **dónde** aparecen:

	Rest	**Spread**
Dónde se usa	En la **definición** de la función (parámetros)	En la **llamada** a la función o dentro de un array/objeto literal
Qué hace	**Agrupa** varios valores sueltos en un array	**Expande** un array en valores sueltos
Dirección	Valores sueltos → array	Array → valores sueltos

```
function sumarTodos(...numeros) {
  // REST: agrupa los argumentos recibidos en un array
  return numeros.reduce((total, n) => total + n, 0);
}

const datos = [1, 2, 3];
console.log(sumarTodos(...datos)); // SPREAD: expande [1, 2, 3] en 1, 2, 3 → 6
```

En este ejemplo se ven los dos a la vez: en la llamada, spread convierte `datos` en tres argumentos; dentro de la función, rest vuelve a reunirlos en el array `numeros`. Son complementarios.

5. Arrays

5.1 Fundamentos de arrays

Un **array** es una colección ordenada de elementos. Cada elemento ocupa una posición identificada por un índice que empieza en `0`; por eso, el primer elemento está en `lista[0]` y el último en `lista[lista.length - 1]`. Los arrays son objetos y pueden contener valores de cualquier tipo, incluso otros arrays. Aunque JavaScript permite mezclar tipos, normalmente conviene guardar en un mismo array valores relacionados y de tipo coherente.

La propiedad `length` indica el límite de posiciones del array, no necesariamente cuántos valores definidos hay. Los arrays son dinámicos: pueden crecer con métodos como `push()` o reducirse ajustando `length`. Reducirla elimina las posiciones que quedan fuera del nuevo límite.

```
const lista = [10, 20, 30];
console.log(lista[0]); // 10
console.log(lista.length); // 3

lista.push(40);
console.log(lista); // [10, 20, 30, 40]

lista.length = 2;
console.log(lista); // [10, 20]
```

Un **hueco** (*hole*) es una posición sin propiedad asignada. No es exactamente lo mismo que guardar el valor `undefined`: la propiedad existe en el segundo caso, pero no en el primero. `length` cuenta ambos tipos de posición.

```
const disperso = [10, , undefined];
console.log(disperso.length); // 3
console.log(1 in disperso); // false: hay un hueco
console.log(2 in disperso); // true: existe y contiene undefined
```

Los índices válidos son enteros no negativos. Acceder a un índice inexistente devuelve `undefined`; asignar a un índice superior a `length` aumenta la longitud y deja huecos intermedios. Una propiedad como `lista[-1]` no es un índice de array y no cambia `length`.

```
const numeros = [10, 20, 30];
console.log(numeros[8]); // undefined

numeros[5] = 60;
console.log(numeros.length); // 6
console.log(3 in numeros); // false: quedó un hueco
```

Las cadenas también permiten leer caracteres por índice (`texto[0]`), pero son inmutables y no son arrays: no tienen métodos como `push()` o `map()`. Si se necesita un array de caracteres, se puede usar el operador spread: `[..."Hola"]`.

Los arrays tipados, como `Int8Array` o `Uint8Array`, son estructuras especializadas de tamaño fijo para datos numéricos binarios. No son arrays normales y se emplean en casos como gráficos o procesamiento de archivos; en los ejercicios iniciales de este tema se utilizarán arrays comunes.

5.1.1 Creación de Arrays

- **Literal de Corchetes**: Es la forma más común y simple de crear un array.

```
const vacio = []; // Array vacío
const lista = [1, 2, 3, 4, 5];
```

- **Constructor `Array`**: Puede crear un array vacío, uno con una longitud determinada o uno con elementos iniciales. `new Array(3)` crea tres huecos, mientras que `new Array(1, 2, 3)` crea un array con esos tres valores.

```
const milista = new Array(); // Array vacío
const otralista = new Array(100); // Array de 100 posiciones (disperso)
const elementos = new Array(1, 2, 3, "hola"); // Array con elementos
iniciales
```

- **Método `Array.of()`**: Crea un array con los elementos pasados como argumento, incluso si es uno solo. Así se evita la particularidad de `new Array(3)`, que crea tres huecos en vez del número `3`.

```
const uno = Array.of(10); // [10]
const varios = Array.of(1, 2, 3); // [1, 2, 3]
const tresHuecos = new Array(3); // length es 3, pero no hay valores
definidos
```

- **Método `Array.from()`**: Crea un array nuevo a partir de un iterable o de un objeto similar a un array. También puede transformar cada elemento mientras lo copia.

```
const copia = Array.from([1, 2, 3, 4]);
const letras = Array.from("Prueba");
const cuadrados = Array.from([1, 2, 3], (numero) => numero ** 2);
console.log(letras); // ["P", "r", "u", "e", "b", "a"]
console.log(cuadrados); // [1, 4, 9]
```

5.1.2 Operador Spread (...)

El operador spread permite expandir un array en sus elementos individuales, lo que es útil para combinar arrays o copiar elementos.

```
const a = [1, 2, 3];
const b = [4, 5, 6];
const c = [...a, ...b]; // [1, 2, 3, 4, 5, 6]
const cadena = "Prueba";
const d = [...cadena]; // ['P', 'r', 'u', 'e', 'b', 'a']
const referencia = a; // Es el mismo array, no una copia
const copia = [...a]; // Copia del array
copia[0] = 100;
console.log(a); // [1, 2, 3] (no cambia)
referencia[0] = 9;
console.log(a); // [9, 2, 3]: referencia y a son el mismo array
```

La copia con spread es **superficial**: si un elemento es otro array u objeto, la copia y el original siguen compartiendo ese elemento anidado.

5.1.3 Leer y escribir elementos

Los elementos se leen y escriben con corchetes y un índice numérico. Los índices empiezan en cero; leer un índice fuera de rango devuelve `undefined`.

- **Acceso a Elementos**: Puedes acceder a un elemento de un array utilizando su índice, que es un número entero positivo. El índice comienza en 0 para el primer elemento.

  ```
  const a = [1, 2, 3, 4, 5];
  console.log(a[0]); // 1
  ```

- **Modificación de Elementos**: Para modificar un valor en un array, simplemente se asigna un nuevo valor al índice correspondiente.

  ```
  a[0] = 0;
  console.log(a); // [0, 2, 3, 4, 5]
  ```

- **Acceso a índices no existentes**: Si intentas leer un índice que no existe, JavaScript devuelve `undefined`.

  ```
  console.log(a[100]); // undefined
  ```

- El operador `in` permite comprobar si existe una posición concreta. Esta comprobación también distingue un hueco de una posición que contiene `undefined`.

```
const valores = [10, , undefined];
console.log(0 in valores); // true
console.log(1 in valores); // false
console.log(2 in valores); // true
```

- **Cambiar `length`**: Aumentar `length` crea huecos; reducirla elimina los elementos que quedan fuera del nuevo límite.

```
const letras = ["a", "b", "c"];
letras.length = 5;
console.log(letras.length); // 5; las dos posiciones nuevas son huecos
letras.length = 2;
console.log(letras); // ["a", "b"]; se eliminó "c"
```

- **Añadir Elementos**:
  - **push()**: Añade uno o más elementos al final del array.
  - **unshift()**: Añade uno o más elementos al inicio del array.

```
const a = [1, 2, 3, 4];
a.push(5); // Añade al final: [1, 2, 3, 4, 5]
a.unshift(0); // Añade al inicio: [0, 1, 2, 3, 4, 5]
console.log(a);
```

- **Eliminar Elementos**:
  - **pop()**: Elimina y retorna el último elemento del array.
  - **shift()**: Elimina y retorna el primer elemento del array.

```
const ultimo = a.pop(); // Elimina 5, retorna 5
const primero = a.shift(); // Elimina 0, retorna 0
console.log(a); // [1, 2, 3, 4]
```

- **Evita `delete` para eliminar elementos**: `delete` no reduce `length`; deja un hueco en la posición. Para quitar elementos y desplazar el resto, utiliza `splice()` o, según el caso, `pop()` y `shift()`.

```
const a = [1, 2, 3, 4, 5];
delete a[2];
console.log(a.length); // 5
console.log(2 in a); // false: ahora hay un hueco
a.splice(2, 1); // Quita una posición y desplaza los elementos siguientes
console.log(a); // [1, 2, 4, 5]
```

- En un array disperso, un bucle `for...of` visita cada índice y produce `undefined` para un hueco. Si se quiere distinguir huecos de valores `undefined`, se puede comprobar el índice con `in`. En cambio, métodos como `forEach()` omiten los huecos.

```
const c = [1, 2, 3, , , 5, 6];
// for...of visita todos los índices; un hueco se lee como undefined
for (const [indice, value] of c.entries()) {
  if (indice in c) {
    console.log(value); // Imprime los valores de las posiciones existentes
  }
}
```

5.1.4 Arrays multidimensionales

JavaScript no tiene un tipo especial de matriz: se representa con un array cuyos elementos son, a su vez, arrays. Cada array interior corresponde a una fila; sus elementos son las columnas.

```
const matriz = [
  [1, 2, 3], // fila 0
  [4, 5, 6], // fila 1
  [7, 8, 9], // fila 2
];

console.log(matriz[1][2]); // 6: fila 1, columna 2
```

Los índices empiezan en `0`. Por eso, `matriz[1]` obtiene la segunda fila y, al añadir `[2]`, se obtiene el elemento de la tercera columna de esa fila. En general, se accede a una celda con `matriz[fila][columna]`. Como cada fila es un array independiente, puede tener una longitud distinta si la estructura no necesita ser rectangular.

Para recorrer todos los elementos, se puede usar un `for` para las filas y otro para las columnas de cada fila:

```
for (let fila = 0; fila < matriz.length; fila++) {
  for (let columna = 0; columna < matriz[fila].length; columna++) {
    console.log(matriz[fila][columna]);
  }
}
```

El bucle exterior visita cada fila. El interior recorre sus columnas; se usa `matriz[fila].length` para que también funcione si las filas tienen longitudes distintas.

5.1.5 Métodos iteradores de Array

Los métodos iteradores reciben una función que se ejecuta con el valor de cada elemento y, en muchos casos, también con su índice y el array original. Los huecos de un array disperso se omiten. La elección del método depende del resultado buscado: recorrer (`forEach`), transformar (`map`), seleccionar (`filter`), buscar (`find`) etc.

La función callback puede recibir `(valor, indice, array)`. Por ejemplo, `map()` y `filter()` devuelven arrays nuevos, `find()` devuelve el primer valor que cumple la condición. En un array vacío, `find()` devuelve `undefined`.

```
console.log(Object.getPrototypeOf([]) === Array.prototype); // true
```

En JavaScript, los métodos iteradores de arrays permiten realizar operaciones sobre cada elemento de un array. Estos métodos toman una función como argumento y la ejecutan para cada elemento del array. Es importante notar que en arrays dispersos, estos métodos no se ejecutan para los elementos vacíos.

1. **`forEach`**: Iterar sobre cada elemento `forEach` ejecuta una función para cada elemento, pero su resultado no es un array nuevo. Puede usarse para mostrar valores o actualizar una variable externa; si se necesita transformar la colección, suele ser más claro usar `map`.

   ```
   const datos = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
   let suma = 0;
   datos.forEach((valor) => {
     suma += valor;
   });
   console.log(suma); // 55
   datos.forEach((valor, indice, array) => {
     array[indice] = valor * 2;
   });
   console.log(datos); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
   ```

2. **`map`**: Crear un nuevo array transformado `map` crea un nuevo array con los resultados de llamar a la función proporcionada en cada elemento del array original.

   ```
   const nuevo = datos.map((valor) => valor / 2);
   console.log(nuevo); // [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
   ```

3. **`filter`**: Filtrar elementos según una condición `filter` crea un nuevo array con todos los elementos que pasen la condición implementada en la función proporcionada.

   ```
   const menores = datos.filter((valor) => valor <= 5);
   console.log(menores); // [2, 4]
   const pares = datos.filter((valor) => valor % 2 === 0);
   console.log(pares); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
   ```

4. **find y findIndex**: Buscar elementos find retorna el primer elemento que cumple la condición especificada en la función proporcionada. findIndex retorna el índice del primer elemento que cumple la condición.

```
const buscado = datos.find((valor) => valor < 5);
console.log(buscado); // 2
const indiceBuscado = datos.findIndex((valor) => valor < 5);
console.log(indiceBuscado); // 0
```

5. **every y some**: Verificar condiciones every verifica si todos los elementos del array cumplen una condición y retorna `true` o `false`. some verifica si al menos un elemento cumple la condición.

```
const todos = datos.every((valor) => valor > 10);
console.log(todos); // false
const alguno = datos.some((valor) => valor > 10);
console.log(alguno); // true
```

> Es importante dominar los métodos iteradores de arrays, ya que son fundamentales para un estilo de programación funcional y para el manejo de datos en general.

5.1.6 Otros Métodos Útiles de Arrays

Estos métodos resuelven tareas frecuentes. Algunos crean un array nuevo (`concat`, `slice`) y otros modifican el array original (`splice`, `fill`, `reverse`, `sort`); consulta esa diferencia antes de usarlos.

Método	Para qué sirve
`concat(otro)`	Combina arrays y devuelve uno nuevo.
`slice(inicio, fin)`	Copia una parte sin incluir el índice `fin`; no modifica el original.
`splice(inicio, cantidad, ...elementos)`	Elimina, inserta o sustituye elementos modificando el original.
`fill(valor)`	Sustituye los valores de un tramo del array por un mismo valor; modifica el original.
`copyWithin(destino, inicio, fin)`	Copia un tramo a otra posición del mismo array; lo modifica.
`indexOf(valor)` / `lastIndexOf(valor)`	Devuelven el primer/último índice del valor, o `-1` si no aparece.
`includes(valor)`	Indica con `true` o `false` si el valor está presente.
`sort()`	Ordena y modifica el array; para números se debe indicar una función de comparación.
`reverse()`	Invierte y modifica el orden del array.
`join(separador)`	Une los elementos en una cadena usando el separador indicado.
`toString()`	Convierte los elementos en una cadena separados por comas.
`Array.isArray(valor)`	Comprueba si un valor es un array.

```
const valores = [12, 3, 25, 8];
const copiaOrdenada = [...valores].sort((a, b) => a - b);
const primerosDos = valores.slice(0, 2);

console.log(copiaOrdenada); // [3, 8, 12, 25]
console.log(primerosDos); // [12, 3]
console.log(valores.includes(25)); // true
console.log(valores.join(" - ")); // "12 - 3 - 25 - 8"
console.log(Array.isArray(valores)); // true
```

Sin función de comparación, `sort()` ordena los valores como cadenas; por eso, por ejemplo, `[2, 10].sort()` produce `[10, 2]`. La función `(a, b) => a - b` especifica un orden numérico ascendente.

6. Objetos

La programación orientada a objetos (POO) organiza un programa alrededor de **objetos**: entidades que reúnen datos y operaciones relacionadas. En JavaScript, los objetos se pueden crear directamente con literales y también mediante **clases**, una sintaxis que permite definir de forma reutilizable cómo crear objetos con el mismo tipo de datos y comportamiento.

En este apartado veremos cómo modelar objetos y cómo se relacionan los conceptos principales de la POO con el modelo basado en prototipos de JavaScript.

6.1 Objetos: estado y comportamiento

Un **objeto** permite representar una cosa del programa y reunir en un mismo lugar:

- sus **datos**, que se guardan en propiedades;
- las **acciones** que puede realizar, que se escriben como métodos.

Por ejemplo, podemos representar una lámpara. Su propiedad `encendida` indica si está encendida o apagada. Esa información es su **estado**. Los métodos `encender()` y `apagar()` son acciones que modifican ese estado; por eso decimos que forman parte de su **comportamiento**.

```
const lampara = {
  encendida: false,

  encender() {
    this.encendida = true;
  },

  apagar() {
    this.encendida = false;
  }
};

console.log(lampara.encendida); // false: empieza apagada
```

```
lampara.encender();
console.log(lampara.encendida); // true: ahora está encendida

lampara.apagar();
console.log(lampara.encendida); // false: vuelve a estar apagada
```

En el ejemplo, `encendida` es una **propiedad**: tiene un nombre y un valor. `encender()` y `apagar()` son **métodos**: son funciones definidas dentro del objeto. Al ejecutarlos con `lampara.encender()`, cambian la propiedad del mismo objeto.

¿Qué significa `this`?

Dentro de `encender()` y `apagar()`, `this` se refiere al objeto que recibe la llamada. Por ejemplo, al ejecutar `lampara.encender()`, `this` representa a `lampara`. Por eso, `this.encendida = true` cambia la propiedad `encendida` de esa lámpara.

La notación de punto permite leer una propiedad (`lampara.encendida`) o ejecutar un método (`lampara.encender()`). También se puede usar la notación de corchetes:

```
const propiedad = "encendida";
console.log(lampara[propiedad]); // false
```

Aquí el nombre de la propiedad está guardado en la variable `propiedad`. Los corchetes permiten usar ese valor para elegir qué propiedad consultar.

Los objetos se comparten por referencia

Al asignar un objeto a otra variable no se crea otro objeto: las dos variables se refieren al mismo. Por eso, un cambio realizado mediante una de ellas también se ve al consultar la otra.

```
const otraReferencia = lampara;
otraReferencia.encender();

console.log(lampara.encendida); // true
```

Aunque `lampara` se declaró con `const`, sus propiedades sí pueden cambiar. `const` impide asignar un objeto distinto a la variable; no hace que el objeto sea inmutable.

6.2 Clases e instancias

Una **clase** describe una estructura y un comportamiento comunes. Un objeto concreto creado a partir de esa clase es una **instancia**. El constructor permite preparar los datos iniciales de cada instancia cuando se crea con `new`.

```
class Persona {
  constructor(nombre, edad) {
```

```
    this.nombre = nombre;
    this.edad = edad;
  }

  saludar() {
    return `Hola, soy ${this.nombre}`;
  }
}

const persona1 = new Persona("Lucía", 28);
const persona2 = new Persona("Marcos", 34);

console.log(persona1.saludar()); // "Hola, soy Lucía"
console.log(persona2.edad);      // 34
console.log(persona1 instanceof Persona); // true
```

6.2.1 Sintaxis del constructor

El constructor se declara dentro de la clase con la palabra `constructor`, como si fuera un método especial. No se escribe `function` delante y solo puede haber uno por clase.

Parte	Significado
`constructor(nombre, edad)`	Declara el constructor y los parámetros que recibirá.
`new Persona("Lucía", 28)`	Crea una instancia y pasa los argumentos en el mismo orden.
`this.nombre = nombre`	Guarda el valor del parámetro como propiedad de la nueva instancia.

```
class Persona {
  constructor(nombre, edad) {
    this.nombre = nombre; // "Lucía" queda guardado en la propiedad nombre
    this.edad = edad;     // 28 queda guardado en la propiedad edad
  }
}

const persona = new Persona("Lucía", 28);
console.log(persona.nombre); // "Lucía"
```

Al ejecutar `new Persona("Lucía", 28)`, JavaScript crea el objeto y ejecuta el constructor. Dentro de él, `this` representa ese nuevo objeto. Así, cada instancia recibe sus propios valores.

> **Recuerda:** el constructor no se llama directamente; se ejecuta al crear una instancia con `new`. Si no declaras uno, JavaScript proporciona un constructor vacío automáticamente.

Los métodos definidos en la clase, como `saludar()`, se comparten mediante el prototipo.

6.2.2 `this` en los métodos

El valor de `this` depende de cómo se llama a un método. Una llamada como `persona1.saludar()` establece `this` como `persona1`. Si se separa el método del objeto, esa relación se pierde:

```
const saludar = persona1.saludar;
// saludar(); // No usar así: el método ya no se invoca como persona1.saludar().
```

Si hace falta guardar el método para llamarlo más tarde, se puede enlazar explícitamente el objeto con `bind()`:

```
const saludarPersona1 = persona1.saludar.bind(persona1);
console.log(saludarPersona1()); // "Hola, soy Lucía"
```

6.3 Encapsulación y validación

La **encapsulación** consiste en proteger el estado interno de un objeto y ofrecer operaciones controladas para consultarlo o modificarlo. Los campos privados de JavaScript empiezan con `#` y solo se pueden utilizar dentro de la clase que los declara.

Visibilidad de los miembros en JavaScript

En JavaScript, los miembros de una clase son **públicos por defecto**: sus propiedades y métodos pueden consultarse o utilizarse desde fuera. No hace falta escribir la palabra `public`, y JavaScript no tiene un modificador `protected` nativo.

Para declarar un miembro privado se usa `#` al principio de su nombre. Solo puede acceder a él el código de la clase que lo declara; no es accesible desde fuera ni directamente desde una subclase. Los objetos literales tampoco admiten modificadores de visibilidad.

```
class CuentaBancaria {
  #saldo = 0;

  constructor(titular, saldoInicial = 0) {
    if (saldoInicial < 0) {
      throw new RangeError("El saldo inicial no puede ser negativo.");
    }

    this.titular = titular;
    this.#saldo = saldoInicial;
  }

  get saldo() {
    return this.#saldo;
  }

  ingresar(cantidad) {
    if (cantidad <= 0) {
      throw new RangeError("La cantidad debe ser mayor que cero.");
```

```
    }

    this.#saldo += cantidad;
  }

  retirar(cantidad) {
    if (cantidad <= 0 || cantidad > this.#saldo) {
      throw new RangeError("La cantidad no es válida para esta retirada.");
    }

    this.#saldo -= cantidad;
  }
}

const cuenta = new CuentaBancaria("Lucía", 100);
cuenta.ingresar(25);
cuenta.retirar(40);
console.log(cuenta.saldo); // 85
// console.log(cuenta.#saldo); // Error: el campo es privado.
```

El saldo no se puede modificar directamente desde fuera de la clase. Los métodos `ingresar()` y `retirar()` comprueban las operaciones antes de cambiarlo. El getter `saldo` permite consultar su valor sin exponer el campo privado.

Un getter se usa como una propiedad (`cuenta.saldo`), aunque se ejecuta como una función. También puede definirse un setter si se necesita permitir una asignación controlada; no es obligatorio añadirlo cuando una operación con nombre, como `ingresar()`, expresa mejor la regla del dominio.

6.4 Miembros estáticos

Los miembros **estáticos** pertenecen a la clase, no a cada instancia. Se declaran con `static` y resultan útiles para operaciones relacionadas con el tipo en su conjunto o para crear instancias con una regla común.

```
class ConversorTemperatura {
  static escala = "Celsius a Fahrenheit";

  static celsiusAFahrenheit(celsius) {
    return (celsius * 9) / 5 + 32;
  }
}

console.log(ConversorTemperatura.escala); // "Celsius a Fahrenheit"
console.log(ConversorTemperatura.celsiusAFahrenheit(20)); // 68
```

Se invocan mediante la clase (`ConversorTemperatura.celsiusAFahrenheit(...)`), no mediante una instancia.

6.5 Herencia

La **herencia** permite crear una clase especializada a partir de otra. La clase general se denomina clase base o padre; la especializada, clase derivada o hija. En JavaScript se utiliza `extends`, y el constructor de la clase derivada debe llamar a `super(...)` antes de usar `this`.

```
class Empleado {
  constructor(nombre, salario) {
    this.nombre = nombre;
    this.salario = salario;
  }

  describirTrabajo() {
    return `${this.nombre} realiza tareas de la empresa`;
  }
}

class Desarrollador extends Empleado {
  constructor(nombre, salario, lenguaje) {
    super(nombre, salario);
    this.lenguaje = lenguaje;
  }

  describirTrabajo() {
    return `${this.nombre} desarrolla software con ${this.lenguaje}`;
  }
}

const desarrollador = new Desarrollador("Marcos", 32000, "JavaScript");
console.log(desarrollador.nombre); // "Marcos", heredado de Empleado
console.log(desarrollador.describirTrabajo());
// "Marcos desarrolla software con JavaScript"
```

`super(...)` ejecuta el constructor de la clase base. En otros métodos, `super.metodo()` permite llamar a la implementación heredada de un método cuando la clase derivada quiere ampliarla:

```
class DesarrolladorSenior extends Desarrollador {
  describirTrabajo() {
    return `${super.describirTrabajo()} y revisa código`;
  }
}
```

Tipos de herencia

Según cómo se relacionan las clases, se suelen distinguir varios tipos:

Tipo	¿Qué significa?	¿Se puede hacer con clases de JavaScript?
Simple	Una clase hija hereda directamente de una clase	Sí. `Desarrollador extends Empleado` es un ejemplo.

Tipo	¿Qué significa?	¿Se puede hacer con clases de JavaScript?
	padre.	
Múltiple	Una clase hija hereda directamente de varias clases padre.	No. Una clase solo puede tener una clase después de `extends`.
Multinivel	Una clase hereda de otra que, a su vez, hereda de una tercera.	Sí. `DesarrolladorSenior` hereda de `Desarrollador`, que hereda de `Empleado`.
Jerárquica	Varias clases hijas heredan de la misma clase padre.	Sí. `Desarrollador` y `Disenador` pueden heredar de `Empleado`.
Híbrida	Combina varios tipos de relación de herencia.	Puede construirse combinando herencia simple, multinivel y jerárquica, pero no habilita la herencia múltiple.

JavaScript permite encadenar prototipos, pero cada clase derivada tiene una sola clase base. Para reutilizar capacidades de varias fuentes, se puede preferir la **composición** (un objeto contiene y utiliza otros objetos) o aplicar patrones como los *mixins*; no es herencia múltiple nativa.

La herencia expresa una relación «es un tipo de». Conviene utilizarla cuando la clase derivada realmente puede tratarse como la clase base, no solo para reutilizar unas pocas líneas de código.

Clases abstractas e interfaces en JavaScript

JavaScript no tiene palabras clave nativas `abstract` ni `interface`. Por tanto, el lenguaje no puede impedir automáticamente que se cree una instancia de una clase abstracta ni comprobar que una clase implemente una interfaz.

Se puede **imitar** una clase abstracta con comprobaciones en tiempo de ejecución, por ejemplo, lanzando un error si se intenta crear la clase base directamente o si un método que debía redefinirse sigue sin implementación. Aun así, JavaScript no obliga a las clases derivadas a implementar esos métodos.

Una interfaz suele representarse como un **contrato de comportamiento**: una función recibe cualquier objeto que ofrezca los métodos que necesita. Puede comprobarse ese requisito cuando se ejecuta el programa:

```
function arrancarVehiculo(vehiculo) {
  if (typeof vehiculo.arrancar !== "function") {
    throw new TypeError("El vehículo debe tener un método arrancar().");
  }

  return vehiculo.arrancar();
}
```

Esta comprobación se hace en tiempo de ejecución; no es una declaración formal de interfaz ni se verifica antes de ejecutar el código.

6.6 Polimorfismo

El **polimorfismo** permite utilizar una misma operación con objetos de tipos distintos, dejando que cada objeto proporcione su propia implementación. En el ejemplo, ambas clases responden a `describirTrabajo()`, pero cada una describe su trabajo de forma diferente.

```
class Disenador extends Empleado {
  describirTrabajo() {
    return `${this.nombre} diseña interfaces`;
  }
}

const equipo = [
  new Desarrollador("Marcos", 32000, "JavaScript"),
  new Disenador("Lucía", 30000)
];

for (const empleado of equipo) {
  console.log(empleado.describirTrabajo());
}
```

El bucle no necesita comprobar el tipo concreto de cada elemento: llama al mismo método y cada instancia ejecuta su versión. Esto facilita ampliar el programa con nuevos tipos de empleado.

6.7 Composición

La **composición** consiste en construir un objeto combinando otros objetos que colaboran entre sí. Es una alternativa a la herencia cuando la relación no es «es un tipo de», o cuando se quiere reutilizar comportamientos sin crear una jerarquía rígida.

```
class Motor {
  arrancar() {
    return "Motor en marcha";
  }
}

class Coche {
  constructor(motor) {
    this.motor = motor;
  }

  arrancar() {
    return this.motor.arrancar();
  }
}

const motor = new Motor();
const coche = new Coche(motor);
console.log(coche.arrancar()); // "Motor en marcha"
```

`Motor` es la clase; `new Motor()` crea un objeto (una instancia) de esa clase. Ese objeto se pasa como argumento a `new Coche(motor)`, y el constructor de `Coche` guarda la misma referencia en `this.motor`. Así, cada coche tiene acceso a un objeto motor y puede delegarle la operación `arrancar()`.

JavaScript no exige que el argumento se haya creado específicamente con `new Motor()`: no hay una declaración de tipo que lo imponga. `Coche` solo necesita que el objeto recibido tenga un método `arrancar()`. Por eso se puede pasar otro objeto compatible y cambiar la implementación del motor sin modificar `Coche`; esta colaboración entre objetos es la composición.

6.8 Conceptos clave

- **Objeto**: reúne propiedades (estado) y métodos (comportamiento).
- **Clase**: define una estructura y un comportamiento comunes para crear instancias.
- **Instancia**: objeto concreto creado a partir de una clase mediante `new`.
- **Constructor**: inicializa una instancia; `this` representa el objeto que se está construyendo.
- **Encapsulación**: protege el estado interno y valida los cambios; `#campo` declara un campo privado.
- **Miembro estático**: pertenece a la clase y se usa sin crear una instancia.
- **Herencia**: especializa una clase con `extends`; `super()` inicializa la parte heredada.
- **Polimorfismo**: permite usar la misma operación con implementaciones distintas.
- **Composición**: construye objetos haciendo que colaboren otros objetos.
- **Prototipo**: objeto en el que JavaScript busca propiedades que la instancia no tiene directamente.

Como criterio práctico, modela con clases cuando necesites crear varias instancias con reglas y operaciones compartidas; para datos simples y puntuales, un objeto literal suele ser suficiente. Prefiere composición cuando represente mejor la relación entre las entidades, y encapsula las reglas importantes para evitar estados inválidos.

7. JSON

JSON (*JavaScript Object Notation*) es un formato de texto para representar datos estructurados. Aunque su sintaxis se inspira en los objetos literales de JavaScript, JSON es independiente del lenguaje: se utiliza habitualmente para intercambiar datos entre un frontend, un backend y otros sistemas.

Un documento JSON puede contener un objeto, un array o un valor simple. Sus valores permitidos son:

- Objetos, delimitados por llaves: `{ "nombre": "Ada" }`.
- Arrays, delimitados por corchetes: `[1, 2, 3]`.
- Cadenas, números, booleanos (`true` y `false`) y `null`.

Por ejemplo, este es un JSON válido que representa datos de un libro:

```
{
  "titulo": "JavaScript moderno",
  "paginas": 320,
  "disponible": true,
  "autores": ["Ana", "Luis"],
```

```
  "editorial": null
}
```

7.1 Sintaxis JSON y objetos de JavaScript

JSON no es exactamente lo mismo que un objeto literal de JavaScript. En JSON, las claves y las cadenas deben escribirse siempre con comillas dobles, no se permiten comentarios ni comas finales, y solo se admiten los valores indicados anteriormente.

```
// Objeto literal de JavaScript: válido en JavaScript, pero no es JSON.
const libro = {
  titulo: "JavaScript moderno",
  categoria: undefined,
};

// Texto JSON válido.
const libroJSON = `{
  "titulo": "JavaScript moderno",
  "paginas": 320
}`;
```

No forman parte del formato JSON `undefined`, las funciones, los símbolos, `NaN`, `Infinity` ni las instancias de clases. Al enviar datos a una API conviene construir objetos con datos simples y validar el formato esperado por el servidor.

7.2 Serialización con `JSON.stringify()`

Serializar significa convertir un valor de JavaScript en una cadena JSON. Se realiza con `JSON.stringify(valor)`. Esta cadena puede enviarse en el cuerpo de una petición HTTP, guardarse en `localStorage` o persistirse en un archivo o una base de datos.

```
const libro = {
  titulo: "JavaScript moderno",
  paginas: 320,
  disponible: true,
  etiquetas: ["web", "programacion"],
};

const textoJSON = JSON.stringify(libro);

console.log(textoJSON);
// {"titulo":"JavaScript moderno","paginas":320,"disponible":true,"etiquetas":
["web","programacion"]}
console.log(typeof textoJSON); // "string"
```

Los valores de un objeto se transforman de forma predecible:

- Las propiedades cuyo valor es `undefined`, una función o un símbolo se omiten.
- Esos mismos valores, cuando aparecen dentro de un array, se convierten en `null`.
- `NaN` e `Infinity` se convierten en `null`.
- Una fecha se serializa como una cadena ISO mediante su método `toJSON()`.

```
const datos = {
  nombre: "Ada",
  telefono: undefined,
  creadoEl: new Date("2026-01-15T10:30:00Z"),
  calcular: () => 2 + 2,
};

console.log(JSON.stringify(datos));
// {"nombre":"Ada","creadoEl":"2026-01-15T10:30:00.000Z"}
```

7.2.1 Elegir o transformar propiedades con `replacer`

El segundo argumento de `JSON.stringify()` es opcional. Puede ser un array de nombres de propiedades que se deben incluir o una función que transforme cada valor durante la serialización.

```
const usuario = {
  id: 7,
  nombre: "Marta",
  contrasena: "no-se-debe-enviar",
};

const usuarioPublico = JSON.stringify(usuario, ["id", "nombre"]);
console.log(usuarioPublico); // {"id":7,"nombre":"Marta"}
```

Una función *replacer* recibe la clave y el valor actual. Por ejemplo, puede ocultar propiedades sensibles antes de registrar o transmitir un objeto:

```
const usuarioSeguro = JSON.stringify(usuario, (clave, valor) => {
  return clave === "contrasena" ? undefined : valor;
});

console.log(usuarioSeguro); // {"id":7,"nombre":"Marta"}
```

7.2.2 Dar formato al resultado

El tercer argumento, `space`, añade sangría al texto resultante. Es útil para mostrar, depurar o guardar JSON legible por personas; no es necesario para enviarlo por red, porque aumenta su tamaño.

```
const textoLegible = JSON.stringify(libro, null, 2);
console.log(textoLegible);
```

```
/*
{
  "titulo": "JavaScript moderno",
  "paginas": 320,
  "disponible": true,
  "etiquetas": [
    "web",
    "programacion"
  ]
}
*/
```

Los objetos también pueden definir un método `toJSON()` para controlar cómo se representan al serializarlos. `Date` utiliza este mecanismo de forma nativa.

7.3 Deserialización con `JSON.parse()`

Deserializar es convertir una cadena JSON válida en un valor de JavaScript. Se realiza con `JSON.parse(texto)`.

```
const copia = JSON.parse(textoJSON);

console.log(copia.titulo); // "JavaScript moderno"
console.log(Array.isArray(copia.etiquetas)); // true
```

`JSON.parse()` espera texto JSON correcto. Si recibe una cadena mal formada, lanza un `SyntaxError`; por ello, los datos externos deben tratarse como no fiables y el error debe gestionarse explícitamente.

```
function leerLibro(texto) {
  try {
    return JSON.parse(texto);
  } catch (error) {
    console.error("No se ha podido interpretar el JSON:", error.message);
    return null;
  }
}

const libroLeido = leerLibro('{"titulo":"JavaScript moderno"}');
const libroInvalido = leerLibro("{titulo: JavaScript moderno}");
```

> `JSON.parse()` reconstruye objetos y arrays simples, pero no recupera automáticamente los tipos originales. Por ejemplo, una fecha serializada vuelve a ser una cadena y debe convertirse con `new Date(valor)` si se necesita trabajar con ella como fecha.

> La serialización y deserialización permiten transmitir y almacenar datos estructurados. Comprueba siempre qué tipo de dato tienes en cada paso: objeto o array de JavaScript, por un lado, y cadena JSON, por otro.

8. Promesas

8.1 ¿Qué problema resuelven las promesas?

Imagina que pides comida a domicilio. Haces el pedido ahora, pero la comida llegará más tarde. Mientras esperas, puedes hacer otras cosas. Cuando llegue, el pedido habrá terminado bien; si el restaurante no puede prepararlo, habrá terminado con un error.

En JavaScript ocurre algo parecido cuando se realiza una tarea que tarda un tiempo: pedir datos a Internet, leer un archivo o esperar un temporizador. El programa no debería quedarse bloqueado mientras espera. Una **promesa** es un objeto que representa un resultado que estará disponible más adelante: puede ser un resultado correcto o un error.

Por ejemplo, esta operación no entrega los datos inmediatamente:

```
const promesa = fetch("https://jsonplaceholder.typicode.com/todos/1");
console.log(promesa);
```

`fetch` empieza la solicitud y devuelve enseguida una promesa. La respuesta del servidor llegará después. Por eso, si intentas usar los datos justo en la siguiente línea, todavía no los tienes:

```
const respuesta = fetch("https://jsonplaceholder.typicode.com/todos/1");
console.log(respuesta); // No son todavía los datos: es una promesa.
```

La promesa es como un comprobante del pedido: todavía no contiene el resultado final, pero nos permite indicar qué hacer cuando el resultado esté disponible.

8.2 Los tres estados de una promesa

Una promesa solo puede estar en uno de estos tres estados:

1. **Pendiente (`pending`)**: el trabajo todavía no ha terminado.
2. **Cumplida (`fulfilled`)**: el trabajo terminó bien y hay un resultado.
3. **Rechazada (`rejected`)**: el trabajo falló y hay un motivo o error.

Una promesa empieza pendiente y, cuando termina, pasa a cumplida o rechazada. No vuelve atrás ni cambia de resultado después de terminar.

Estado	Qué significa	Ejemplo
Pendiente	Aún estamos esperando	La petición al servidor sigue en curso
Cumplida	La operación salió bien	El servidor envió los datos
Rechazada	La operación falló	No hay conexión a Internet

8.3 Crear una promesa sencilla

Normalmente usarás promesas que ya crean funciones como `fetch`. Aun así, crear una promesa pequeña ayuda a entender cómo funciona.

```
const promesaDelPedido = new Promise((resolve, reject) => {
  const pedidoPreparado = true;

  if (pedidoPreparado) {
    resolve("¡Aquí tienes tu pedido!");
  } else {
    reject(new Error("No se pudo preparar el pedido."));
  }
});
```

Vamos paso a paso:

1. `new Promise(...)` crea la promesa.
2. La función que se pasa a `Promise` se ejecuta para realizar el trabajo.
3. `resolve(valor)` significa «todo ha salido bien» y entrega el resultado.
4. `reject(error)` significa «algo ha fallado» y entrega el motivo.

En este ejemplo `pedidoPreparado` vale `true`, así que la promesa terminará cumplida. Para recibir el resultado, usamos `.then(...)`:

```
promesaDelPedido.then(resultado => {
  console.log(resultado);
});
```

La consola mostrará:

```
¡Aquí tienes tu pedido!
```

Si el trabajo puede fallar, usamos `.catch(...)` para tratar el error:

```
promesaDelPedido
  .then(resultado => {
    console.log(resultado);
  })
  .catch(error => {
    console.error("El pedido ha fallado:", error.message);
  });
```

Prueba a cambiar `pedidoPreparado` a `false`. En ese caso se ejecutará `.catch(...)` en vez del código de `.then(...)`.

8.4 Usar `.then()`, `.catch()` y `.finally()`

- `.then(funcion)` indica qué hacer cuando la promesa se cumple. Recibe el resultado.
- `.catch(funcion)` indica qué hacer si la promesa se rechaza. Recibe el error.
- `.finally(funcion)` indica qué hacer cuando la operación termina, tanto si sale bien como si falla. No recibe el resultado.

```
promesaDelPedido
  .then(resultado => {
    console.log("Éxito:", resultado);
  })
  .catch(error => {
    console.error("Error:", error.message);
  })
  .finally(() => {
    console.log("El pedido ya ha terminado.");
  });
```

`finally` es útil para tareas de limpieza, como ocultar un indicador de carga. No sustituye a `then` ni a `catch`: no sirve para decidir si el resultado fue correcto o fallido.

8.5 Esperar un poco: promesa con un temporizador

Este ejemplo simula una tarea que tarda dos segundos:

```
function prepararPedido() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("Pedido preparado");
    }, 2000);
  });
}

console.log("1. Hago el pedido");

prepararPedido().then(resultado => {
  console.log("3.", resultado);
});

console.log("2. Mientras espero, sigo con otra tarea");
```

El orden que verás será:

```
1. Hago el pedido
2. Mientras espero, sigo con otra tarea
3. Pedido preparado
```

Aunque la promesa tarde, JavaScript no se queda parado: ejecuta las instrucciones que puede y ejecuta el `then` cuando la tarea termina.

8.6 Encadenar promesas

Un `.then(...)` puede devolver un valor. Ese valor se entrega al siguiente `.then(...)`. Así puedes hacer varias operaciones en orden, sin anidar una función dentro de otra:

```
prepararPedido()
  .then(mensaje => {
    console.log(mensaje);
    return "Pedido enviado";
  })
  .then(mensaje => {
    console.log(mensaje);
  })
  .catch(error => {
    console.error("No se pudo completar el pedido:", error.message);
  });
```

Piensa en la cadena como una cinta transportadora:

1. La primera promesa termina y entrega un valor.
2. El primer `.then` recibe ese valor y devuelve otro.
3. El siguiente `.then` recibe lo que devolvió el anterior.
4. Si ocurre un error en cualquier paso, se puede tratar en el `.catch`.

Si dentro de un `.then` empiezas otra tarea asíncrona, devuelve su promesa para que la cadena espere a que termine:

```
obtenerPedido()
  .then(pedido => guardarPedido(pedido))
  .then(resultadoGuardado => {
    console.log("Guardado:", resultadoGuardado);
  })
  .catch(error => {
    console.error("Falló algún paso:", error);
  });
```

8.7 Obtener datos con `fetch`

`fetch` permite hacer solicitudes HTTP, por ejemplo, para pedir datos a una API. Devuelve una promesa, así que usamos `.then(...)` para trabajar con la respuesta cuando llegue.

```
fetch("https://jsonplaceholder.typicode.com/todos/1")
  .then(respuesta => respuesta.json())
  .then(tarea => {
```

```
    console.log("Título:", tarea.title);
    console.log("¿Completada?:", tarea.completed);
  })
  .catch(error => {
    console.error("No se pudo completar la solicitud:", error);
  });
```

Lee el ejemplo en este orden:

1. `fetch(url)` pide el recurso y devuelve una promesa.
2. El primer `.then` recibe un objeto `Response`.
3. `respuesta.json()` lee el cuerpo de la respuesta y también devuelve una promesa. Por eso hay otro `.then`.
4. El segundo `.then` recibe los datos ya convertidos a un objeto JavaScript.
5. `.catch` permite mostrar errores de red u otros errores ocurridos durante la cadena.

Importante: `fetch` normalmente no rechaza la promesa solo porque el servidor responda con un código HTTP de error, como `404` o `500`. Hay que comprobar `respuesta.ok`:

```
fetch("https://jsonplaceholder.typicode.com/todos/1")
  .then(respuesta => {
    if (!respuesta.ok) {
      throw new Error(`Error HTTP: ${respuesta.status}`);
    }

    return respuesta.json();
  })
  .then(tarea => {
    console.log("Datos recibidos:", tarea);
  })
  .catch(error => {
    console.error("No se pudieron obtener los datos:", error.message);
  });
```

Si `respuesta.ok` es `false`, `throw new Error(...)` genera un error y el flujo pasa al `.catch`. Devolver `respuesta.json()` también es importante: así el siguiente `.then` espera a que termine la lectura del cuerpo.

8.8 La misma idea con `async` y `await`

`async` y `await` permiten escribir código que usa promesas con una apariencia más parecida a una lista de instrucciones normal. No hacen que la tarea tarde menos ni convierten el trabajo en síncrono: solo ofrecen otra forma de organizar la espera.

1. Pon `async` delante de la función que tendrá que esperar.
2. Dentro de esa función, escribe `await` delante de la promesa que quieras esperar.
3. Usa `try` y `catch` para tratar posibles errores.

La solicitud anterior se puede escribir así:

```
async function obtenerTarea() {
  try {
    const respuesta = await fetch(
      "https://jsonplaceholder.typicode.com/todos/1"
    );

    if (!respuesta.ok) {
      throw new Error(`Error HTTP: ${respuesta.status}`);
    }

    const tarea = await respuesta.json();
    console.log("Título:", tarea.title);
    console.log("¿Completada?:", tarea.completed);
  } catch (error) {
    console.error("No se pudieron obtener los datos:", error.message);
  }
}

obtenerTarea();
```

`await` significa «espera aquí el resultado de esta promesa antes de continuar con la siguiente instrucción de esta función». Mientras espera, el navegador puede seguir atendiendo otras tareas. `await` se usa dentro de una función `async`.

En este ejemplo:

- `await fetch(...)` espera a que llegue la respuesta HTTP.
- Se comprueba `respuesta.ok` para detectar respuestas HTTP fallidas.
- `await respuesta.json()` espera a que el cuerpo se convierta en datos.
- Si ocurre un error, `catch` lo recibe y lo muestra.

Una función declarada con `async` siempre devuelve una promesa. Por eso, si quieres usar el valor que devuelve, puedes esperarlo desde otra función `async` o usar `.then(...)`:

```
async function saludar() {
  return "Hola";
}

saludar().then(mensaje => {
  console.log(mensaje);
});
```

8.9 Depurar una promesa o una solicitud

Las herramientas de desarrollo del navegador ayudan a averiguar en qué paso se produce un problema.

Para revisar una solicitud de `fetch`:

1. Abre las herramientas de desarrollo del navegador (normalmente con F12).
2. Entra en la pestaña **Network** (Red).
3. Recarga la página o ejecuta la función que realiza la solicitud.
4. Busca la solicitud en la lista y selecciónala.
5. Revisa el estado HTTP, los encabezados, la respuesta y el tiempo de carga.
6. Si la solicitud falla, comprueba la URL, la conexión y el código de estado. Recuerda que un estado `404` o `500` no siempre activa por sí solo el `.catch` de `fetch`; comprueba `ok`.

Para revisar código con `async` y `await`:

1. Pon un breakpoint antes de un `await`.
2. Ejecuta el código y, cuando se pare, comprueba los valores de las variables.
3. Avanza paso a paso (*Step Over*) para continuar sin entrar en detalles internos del navegador.
4. Después del `await`, comprueba el resultado recibido.
5. Si se produce un error, revisa el bloque `catch` y el mensaje de la consola.

8.10 Errores típicos al empezar

- **Intentar usar el resultado inmediatamente:** `fetch(...)` devuelve una promesa, no los datos. Usa `.then(...)` o `await`.
- **Olvidar que `response.json()` también tarda:** devuelve otra promesa; espera su resultado con otro `.then(...)` o con `await`.
- **No devolver una promesa en una cadena:** si un `.then` inicia otra operación asíncrona, devuelve esa promesa para que el siguiente paso espere.
- **Pensar que cualquier estado HTTP activa `catch`:** comprueba `respuesta.ok` y genera un error si la respuesta no es correcta.
- **Olvidar tratar errores:** usa `.catch(...)` o un `try`/`catch` alrededor del código con `await`.
- **Olvidar llamar a la función:** definir `async function obtenerTarea() { ... }` no la ejecuta; hay que llamar a `obtenerTarea()`.

> **Idea clave:** una promesa representa un resultado que llegará más tarde. Usa `.then()` y `.catch()` para reaccionar a ese resultado, o `async` y `await` para escribir la espera de forma más fácil de leer.

9. DOM y eventos

Una página web no es solo el HTML que se descarga: el navegador lo interpreta, construye una representación que JavaScript puede consultar y modificar, y reacciona a las acciones del usuario. El **DOM** permite trabajar con esa representación; los **eventos** permiten responder a lo que sucede en ella.

9.1 DOM

9.1.1 ¿Qué es el DOM?

El **Document Object Model** (DOM, o Modelo de Objetos del Documento) es la representación del documento HTML como un árbol de objetos. Cada etiqueta se convierte en un nodo; los elementos pueden contener otros elementos, texto y atributos.

Por ejemplo, este HTML:

```
<main>
  <h1>Mi página</h1>
  <p>Hola, <strong>mundo</strong>.</p>
</main>
```

se representa, de forma simplificada, así:

```
document
└── html
    └── body
        └── main
            ├── h1
            │   └── "Mi página"
            └── p
                ├── "Hola, "
                ├── strong
                │   └── "mundo"
                └── "."
```

El objeto `document` es el punto de entrada habitual para consultar el documento. Los elementos del árbol tienen propiedades y métodos: se pueden localizar, leer, crear, modificar y eliminar. Los cambios hechos al DOM se reflejan en la página sin tener que volver a cargar el HTML.

El DOM no es el lenguaje JavaScript ni el archivo HTML: es una API del navegador para trabajar con el documento ya interpretado. JavaScript también puede ejecutarse fuera del navegador, donde no existe necesariamente un `document`.

9.1.2 Seleccionar elementos

Antes de modificar un elemento hay que localizarlo. Los selectores CSS permiten buscar por id, clase, etiqueta u otras relaciones:

```
<h1 id="titulo">Catálogo</h1>
<p class="aviso">Elige un elemento.</p>
<p class="aviso">Puedes cambiar la selección.</p>
```

```
const titulo = document.querySelector("#titulo");
const primerAviso = document.querySelector(".aviso");
const todosLosAvisos = document.querySelectorAll(".aviso");
const parrafos = document.getElementsByTagName("p");
```

Método	Resultado
`document.getElementById("titulo")`	El elemento cuyo `id` coincide, o `null` si no existe.

Método	Resultado
`document.querySelector("selector")`	El primer elemento que coincide con un selector CSS, o `null`.
`document.querySelectorAll("selector")`	Una colección estática (`NodeList`) de todos los elementos coincidentes; puede estar vacía.

`querySelector()` y `querySelectorAll()` admiten selectores CSS como `#id`, `.clase`, `button`, `main p` o `form > input`. Hay que comprobar que un resultado puede ser `null` antes de utilizarlo:

```
const boton = document.querySelector("#guardar");

if (boton !== null) {
  boton.textContent = "Guardar cambios";
}
```

`querySelectorAll()` permite recorrer los resultados con `for...of` o `forEach()`:

```
document.querySelectorAll(".aviso").forEach(aviso => {
  console.log(aviso.textContent);
});
```

9.1.3 Leer y cambiar el contenido

Estas propiedades no son intercambiables:

- **`textContent`** lee o cambia el texto de un elemento. El texto asignado se trata como texto, no como etiquetas HTML.
- **`innerHTML`** lee o reemplaza el contenido interpretándolo como HTML.
- **`innerText`** representa el texto visible y tiene en cuenta el estilo y la presentación.

```
const titulo = document.querySelector("#titulo");

if (titulo !== null) {
  titulo.textContent = "Pokédex";
  console.log(titulo.textContent);
}
```

Para mostrar texto procedente de una API, un formulario o una persona, prefiere `textContent`. Insertar ese dato concatenándolo en `innerHTML` puede interpretar texto malicioso como HTML. Reserva `innerHTML` para plantillas HTML controladas y nunca lo uses con contenido no confiable.

También se pueden consultar o cambiar atributos y propiedades:

```
const imagen = document.querySelector("img");

if (imagen !== null) {
  imagen.setAttribute("alt", "Imagen del Pokémon");
  imagen.src = "pokemon.png";
  console.log(imagen.getAttribute("alt"));
}
```

Algunos atributos corresponden a propiedades del elemento. Por ejemplo, en un campo de texto, `input.value` contiene el valor actual escrito por el usuario; `input.getAttribute("value")` se refiere al atributo HTML inicial y no siempre refleja los cambios posteriores.

9.1.4 Crear, añadir y eliminar elementos

Para construir una interfaz desde JavaScript se crea cada nodo y se añade al árbol:

```
const lista = document.querySelector("#lista");

if (lista !== null) {
  const elemento = document.createElement("li");
  elemento.textContent = "Pikachu";
  lista.append(elemento);
}
```

Método	Uso habitual
`document.createElement("li")`	Crear un elemento nuevo.
`element.append(contenido)`	Añadir al final uno o más nodos o cadenas.
`element.prepend(contenido)`	Añadir al principio.
`element.replaceChildren(...)`	Vaciar el contenido y añadir los nodos indicados.
`element.remove()`	Eliminar el elemento del documento.
`element.before(...)` / `element.after(...)`	Insertar contenido antes o después de un elemento.

Para generar una lista, limpia el contenedor una vez y añade un nodo por cada resultado:

```
lista.replaceChildren();

for (const nombre of ["Pikachu", "Bulbasaur", "Charmander"]) {
  const opcion = document.createElement("li");
  opcion.textContent = nombre;
  lista.append(opcion);
}
```

9.1.5 Clases, estilos y relaciones entre nodos

La propiedad `element.style` permite cambiar estilos CSS directamente desde JavaScript. Los nombres de propiedades CSS con guiones se escriben en *camelCase* (`background-color` pasa a `backgroundColor`) y los valores se asignan como cadenas:

```
const titulo = document.querySelector("h1");

if (titulo !== null) {
  titulo.style.color = "navy";
  titulo.style.backgroundColor = "lightyellow";
  titulo.style.fontSize = "2rem";
  titulo.style.display = "none";
}
```

Para leer o cambiar una propiedad CSS usando su nombre CSS original, también puedes usar `style.setProperty()` y `style.getPropertyValue()`. `removeProperty()` quita una declaración en línea:

```
if (titulo !== null) {
  titulo.style.setProperty("border-color", "tomato");
  console.log(titulo.style.getPropertyValue("border-color"));
  titulo.style.removeProperty("display");
}
```

`element.style` lee y modifica principalmente los estilos en línea del elemento, no todos los estilos que recibe de las hojas CSS. Para consultar el estilo calculado (el valor final aplicado por el navegador), usa `getComputedStyle()`:

```
if (titulo !== null) {
  const estiloCalculado = getComputedStyle(titulo);
  console.log(estiloCalculado.color);
}
```

Para estados visuales habituales sigue siendo preferible definir las reglas en CSS y activar o desactivar una clase desde JavaScript. Así se mantiene separada la presentación del comportamiento y es más fácil cambiar el diseño:

```
const tarjeta = document.querySelector(".tarjeta");

if (tarjeta !== null) {
  tarjeta.classList.add("seleccionada");
  tarjeta.classList.remove("oculta");
  tarjeta.classList.toggle("favorita");
```

```
  console.log(tarjeta.classList.contains("favorita"));
}
```

```
.tarjeta.seleccionada {
  border: 2px solid royalblue;
  background-color: #eef4ff;
}
```

Usa los estilos directos cuando el valor sea realmente dinámico (por ejemplo, una posición o un color calculado). Para estados con nombre —como `oculta`, `seleccionada` o `favorita`— utiliza clases CSS. Para ocultar un elemento, considera también el atributo `hidden` (asigna `element.hidden = true` para ocultarlo o `element.hidden = false` para mostrarlo) si la acción expresa que el contenido deja de estar disponible:

```
const aviso = document.querySelector("#aviso");

if (aviso !== null) {
  aviso.hidden = true;
}
```

Las relaciones entre nodos también sirven para recorrer la página:

- `parentElement`: elemento padre.
- `children`: elementos hijos.
- `firstElementChild` y `lastElementChild`: primer y último elemento hijo.
- `nextElementSibling` y `previousElementSibling`: elemento siguiente y anterior del mismo nivel.

Usa estas relaciones cuando la estructura de HTML sea estable. Para localizar un elemento de forma clara, suele ser más sencillo seleccionar directamente por id o por un selector CSS específico.

9.1.6 Cuándo ejecutar JavaScript

Si el script intenta seleccionar un elemento antes de que el navegador haya leído su HTML, el resultado puede ser `null`. Hay tres patrones habituales:

1. Cargar el archivo externo al final de `<body>`.
2. Añadir `defer` a un script externo; se descarga en paralelo y se ejecuta cuando el HTML ya se ha interpretado.
3. Esperar al evento `DOMContentLoaded` si el código se carga desde el `<head>` o su orden no está controlado.

```
<script src="app.js" defer></script>
```

```
document.addEventListener("DOMContentLoaded", () => {
  const titulo = document.querySelector("h1");
  console.log(titulo?.textContent);
});
```

El operador `?.` evita acceder a una propiedad si el valor de la izquierda es `null` o `undefined`. Es útil cuando la ausencia del elemento es aceptable; si el elemento es obligatorio, conviene comprobarlo y mostrar un error explícito.

9.1.7 Ejemplo práctico: crear una página con el DOM

Aunque el archivo HTML puede tener el `body` vacío, JavaScript puede crear y añadir todos los elementos de la página. Usa etiquetas semánticas y `textContent` para que el texto se trate como texto, no como código HTML.

Guarda estos dos archivos en la misma carpeta. El `body` de `index.html` no contiene elementos: JavaScript construirá la página y también aplicará sus estilos.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Recetas caseras</title>
  <script src="app.js" defer></script>
</head>
<body></body>
</html>
```

En `app.js`, crea una página sencilla con un título y una lista de recetas. Añade algunos estilos directamente desde JavaScript:

```
document.body.style.cssText =
  "margin:0;padding:30px;font-family:Arial,sans-
serif;background:#f3f5ee;color:#243b2b";

const pagina = document.createElement("main");
pagina.style.cssText =
  "max-width:600px;margin:auto;padding:24px;background:white;border-radius:12px";

const titulo = document.createElement("h1");
titulo.textContent = "Mi recetario";
titulo.style.color = "#327044";

const texto = document.createElement("p");
texto.textContent = "Ideas sencillas para cocinar en casa:";
```

```
const lista = document.createElement("ul");
for (const receta of ["Tortilla de patatas", "Ensalada fresca", "Sopa de
verduras"]) {
  const elemento = document.createElement("li");
  elemento.textContent = receta;
  elemento.style.padding = "8px";
  lista.append(elemento);
}

pagina.append(titulo, texto, lista);
document.body.append(pagina);
```

`createElement()` crea cada etiqueta, `append()` la incorpora a su elemento contenedor y `textContent` establece su texto. El bucle `for...of` crea un elemento `<li>` por cada receta. Las propiedades `style` aplican los estilos desde JavaScript. Como el script se carga con `defer`, el navegador ya ha leído el documento cuando se ejecuta.

9.2 Eventos

9.2.1 ¿Qué es un evento?

Un **evento** es una notificación de que ha ocurrido algo en el navegador o en la página: se ha hecho clic, se ha escrito una tecla, se ha enviado un formulario, ha terminado de cargar una imagen o se ha recibido una respuesta de red. El código que se ejecuta para reaccionar se llama **manejador** o *handler*.

La forma recomendada de asociar un manejador es `addEventListener()`:

```
const boton = document.querySelector("#saludar");

if (boton !== null) {
  boton.addEventListener("click", () => {
    console.log("Se ha pulsado el botón");
  });
}
```

Su forma general es:

```
elemento.addEventListener("tipoDeEvento", manejador);
```

El primer argumento es el nombre del evento, sin el prefijo `on`. El segundo es una función que el navegador ejecuta cuando se produce el evento. Se pasa la función, sin llamarla: escribe `() => ...` o `miFuncion`, no `miFuncion()`.

`addEventListener()` es la forma recomendada de registrar eventos en JavaScript nativo. El manejador se puede escribir directamente como una función anónima o se puede definir aparte y pasar su nombre como segundo argumento.

```
// Función anónima: se define en el mismo lugar donde se registra.
boton.addEventListener("click", () => {
  console.log("Se ha pulsado el botón");
});

// Función nombrada: se define aparte y se pasa sin paréntesis.
function mostrarMensaje() {
  console.log("Se ha pulsado el botón");
}

boton.addEventListener("click", mostrarMensaje);
```

En el segundo ejemplo se pasa la función como parámetro: `mostrarMensaje` es una referencia a la función, que el navegador ejecutará cuando ocurra el evento. No escribas `mostrarMensaje()`, porque eso la ejecutaría inmediatamente al registrar el manejador. La función nombrada resulta útil si se quiere reutilizar en varios elementos o quitar más adelante con `removeEventListener()`. Para retirarla, se pasa el mismo tipo de evento y la misma referencia a la función:

```
boton.addEventListener("click", mostrarMensaje);
boton.removeEventListener("click", mostrarMensaje);
```

Evita atributos HTML como `onclick="..."` y asignaciones como `boton.onclick = ...` para código nuevo: `addEventListener()` permite asociar varios manejadores, separar HTML y JavaScript y controlar mejor cuándo se registran.

9.2.2 El objeto del evento y el comportamiento predeterminado

El navegador pasa al manejador un objeto con información sobre lo ocurrido:

```
const campo = document.querySelector("#nombre");

if (campo !== null) {
  campo.addEventListener("input", evento => {
    console.log(evento.type);
    console.log(evento.target.value);
  });
}
```

Propiedades y métodos de uso frecuente:

- `event.type`: nombre del evento, por ejemplo `"click"`.
- `event.target`: nodo donde se originó el evento.
- `event.currentTarget`: elemento al que se registró el manejador.
- `event.preventDefault()`: cancela la acción predeterminada si el evento lo permite.

- `event.stopPropagation()`: evita que el evento siga propagándose; úsalo solo si hay una razón concreta, porque puede interferir con otros manejadores.

Al enviar un formulario, el navegador normalmente intenta navegar o recargar la página. Una aplicación que lo procesa con JavaScript puede evitar ese comportamiento:

```
const formulario = document.querySelector("#busqueda");

if (formulario !== null) {
  formulario.addEventListener("submit", evento => {
    evento.preventDefault();
    console.log("Procesar el formulario sin recargar la página");
  });
}
```

Los eventos suelen **propagarse** por el árbol. En la fase de burbujeo, el evento parte del nodo objetivo y sube por sus antecesores. Por eso un clic en un botón dentro de una tarjeta también puede ser observado desde la tarjeta o desde un contenedor.

Este comportamiento permite la **delegación de eventos**: registrar un solo manejador en un contenedor para atender eventos de muchos elementos hijos, incluso si se crean después:

```
const lista = document.querySelector("#lista");

if (lista !== null) {
  lista.addEventListener("click", evento => {
    if (!(evento.target instanceof Element)) {
      return;
    }

    const opcion = evento.target.closest("li");
    if (opcion !== null && lista.contains(opcion)) {
      console.log("Elemento seleccionado:", opcion.textContent);
    }
  });
}
```

9.2.3 Tipos de eventos habituales

Grupo	Eventos comunes	Cuándo se producen
Ratón y puntero	`click`, `dblclick`, `pointerdown`, `pointerup`, `pointermove`	Al pulsar o mover el puntero. Los eventos `pointer` cubren ratón, lápiz y pantallas táctiles.
Teclado	`keydown`, `keyup`	Al pulsar y soltar teclas. `event.key` indica la tecla, por ejemplo `"Enter"`.

Grupo	Eventos comunes	Cuándo se producen
Formularios	`input`, `change`, `submit`, `focus`, `blur`	Al editar campos, confirmar un cambio, enviar un formulario o cambiar el foco.
Documento y ventana	`DOMContentLoaded`, `load`, `resize`, `scroll`	Al interpretar el HTML, cargar recursos, cambiar el tamaño o desplazar la página.
Portapapeles	`copy`, `cut`, `paste`	Al copiar, cortar o pegar contenido.
Arrastrar y soltar	`dragstart`, `dragover`, `drop`, `dragend`	Durante una operación de arrastrar y soltar.
Multimedia	`play`, `pause`, `ended`, `timeupdate`	Durante la reproducción de audio o vídeo.
Red y recursos	`online`, `offline`, `error`, `load`	Al cambiar la conexión o al cargar/fallar un recurso.

Hay diferencias importantes entre eventos de formularios:

- `input` se produce con cada cambio del valor mientras la persona escribe.
- `change` se produce cuando el valor se confirma; en campos de texto suele ocurrir al perder el foco, y en listas desplegables al seleccionar otra opción.
- `submit` se produce al enviar el formulario, también si se pulsa Enter desde un campo.
- `focus` y `blur` indican que un control recibe o pierde el foco.

Para comportamientos esenciales de una interfaz, no dependas solo de eventos de ratón: utiliza controles HTML apropiados, como `<button>` y `<form>`, que ya admiten teclado y tecnologías de asistencia.

9.2.4 Validación de formularios

La validación comprueba que los datos cumplen las condiciones necesarias antes de procesar un formulario. Conviene declarar primero las reglas sencillas en HTML: el navegador puede validarlas automáticamente y comunicar los problemas al usuario.

Algunos atributos de validación habituales son:

- `required`: el campo no puede quedar vacío.
- `type="email"` y `type="url"`: el valor debe tener el formato correspondiente.
- `min`, `max` y `step`: limitan los valores numéricos o las fechas.
- `minlength`, `maxlength` y `pattern`: limitan la longitud o el formato del texto.

```
<form id="formulario-cafe">
  <label for="id-cafe">Número del café</label>
  <input id="id-cafe" name="id" type="number" min="1" step="1" required>
  <button type="submit">Buscar café</button>
</form>
```

Al enviar este formulario, el navegador comprueba que el campo contiene un número entero mayor que cero. Si el valor no es válido, muestra el aviso correspondiente y no dispara el evento `submit`. Así se evita procesar

datos incorrectos sin tener que escribir desde cero las comprobaciones más comunes.

La API de validación de restricciones permite consultar o mostrar esos errores desde JavaScript:

- `campo.validity.valid` indica si un campo cumple las restricciones.
- `formulario.checkValidity()` comprueba los campos y devuelve `true` o `false`.
- `formulario.reportValidity()` comprueba los campos y pide al navegador que muestre los mensajes de validación.
- `campo.setCustomValidity("mensaje")` establece un error personalizado. Para quitarlo, vuelve a llamar al método con una cadena vacía: `campo.setCustomValidity("")`.

Por ejemplo, si se quiere controlar el envío desde JavaScript, se puede añadir `novalidate` al formulario y realizar la comprobación en el manejador `submit`:

```
<form id="formulario-cafe" novalidate>
  <label for="id-cafe">Número del café</label>
  <input id="id-cafe" name="id" type="number" min="1" step="1" required>
  <button type="submit">Buscar café</button>
</form>
```

```
const formulario = document.querySelector("#formulario-cafe");

if (formulario !== null) {
  formulario.addEventListener("submit", evento => {
    evento.preventDefault();

    if (!formulario.checkValidity()) {
      formulario.reportValidity();
      return;
    }

    console.log("Formulario válido: ya se puede procesar");
  });
}
```

`novalidate` desactiva la validación automática al enviar, pero no elimina las reglas declaradas con atributos HTML: `checkValidity()` y `reportValidity()` todavía pueden consultarlas. Úsalo solo si necesitas controlar explícitamente el momento de la validación. Si basta con el comportamiento del navegador, omite `novalidate` y deja que el navegador valide antes de emitir `submit`.

Para una regla que HTML no puede expresar, usa `setCustomValidity()` y borra el mensaje cuando el valor vuelva a ser válido. Si se deja un mensaje personalizado activo, el campo seguirá siendo inválido aunque su valor cumpla las reglas HTML.

9.2.5 Ejemplos de uso de eventos

Los siguientes ejemplos muestran cómo responder a distintos eventos habituales. Cada fragmento supone que el HTML contiene los elementos con los identificadores indicados.

Ejemplo 1. Actualizar un saludo mientras se escribe

El evento `input` se dispara cada vez que cambia el valor del campo:

```
<label for="nombre">Tu nombre</label>
<input id="nombre" type="text">
<p id="saludo" aria-live="polite"></p>
```

```
const campoNombre = document.querySelector("#nombre");
const saludo = document.querySelector("#saludo");

if (campoNombre !== null && saludo !== null) {
  campoNombre.addEventListener("input", () => {
    // trim() elimina los espacios al principio y al final.
    const nombre = campoNombre.value.trim();
    if (nombre === "") {
      saludo.textContent = "Escribe tu nombre.";
    } else {
      saludo.textContent = `Hola, ${nombre}.`;
    }
  });
}
```

Ejemplo 2. Validar un formulario antes de procesarlo

El formulario tiene cinco campos distintos. `novalidate` permite que el evento `submit` se ejecute aunque haya valores incorrectos, para que JavaScript pueda comprobar cada campo y mostrar los mensajes juntos debajo del formulario.

```
<form id="formulario-registro" novalidate>
  <label for="nombre-registro">Nombre</label>
  <input id="nombre-registro" name="nombre" type="text" minlength="2" required>

  <label for="correo-registro">Correo electrónico</label>
  <input id="correo-registro" name="correo" type="email" required>

  <label for="clave-registro">Contraseña</label>
  <input id="clave-registro" name="clave" type="password" minlength="8" required>

  <label for="edad-registro">Edad</label>
  <input id="edad-registro" name="edad" type="number" min="18" max="120" required>

  <label for="fecha-registro">Fecha de nacimiento</label>
  <input id="fecha-registro" name="fecha" type="date" required>

  <button type="submit">Enviar</button>
```

```
</form>
<p id="errores-registro" role="alert"></p>
```

```
const formulario = document.querySelector("#formulario-registro");
const nombre = document.querySelector("#nombre-registro");
const correo = document.querySelector("#correo-registro");
const clave = document.querySelector("#clave-registro");
const edad = document.querySelector("#edad-registro");
const fecha = document.querySelector("#fecha-registro");
const erroresRegistro = document.querySelector("#errores-registro");

if (
  formulario !== null &&
  nombre !== null &&
  correo !== null &&
  clave !== null &&
  edad !== null &&
  fecha !== null &&
  erroresRegistro !== null
) {
  formulario.addEventListener("submit", evento => {
    evento.preventDefault();

    const errores = [];

    if (nombre.value.trim().length < 2) {
      errores.push("El nombre debe tener al menos 2 caracteres.");
    }

    if (!correo.validity.valid) {
      errores.push("Escribe un correo electrónico válido.");
    }

    if (clave.value.length < 8) {
      errores.push("La contraseña debe tener al menos 8 caracteres.");
    }

    const numeroEdad = Number(edad.value);
    if (!Number.isInteger(numeroEdad) || numeroEdad < 18 || numeroEdad > 120) {
      errores.push("La edad debe ser un número entero entre 18 y 120.");
    }

    if (!fecha.validity.valid) {
      errores.push("Selecciona una fecha de nacimiento.");
    }

    // Muestra todos los errores en el párrafo situado bajo el formulario.
    erroresRegistro.textContent = errores.join(" ");

    if (errores.length === 0) {
      erroresRegistro.textContent = "Formulario válido. Ya se puede procesar.";
```

```
    }
  });
}
```

`novalidate` desactiva los avisos automáticos al enviar, pero las reglas HTML siguen disponibles para comprobarse desde JavaScript. Para una validación más sencilla, puedes quitar `novalidate` y dejar que el navegador impida el envío cuando un campo no sea válido.

Ejemplo 3. Cambiar la apariencia con un clic

El evento `click` puede alternar una clase CSS para mostrar un estado visual. El botón sigue siendo un control semántico y se puede activar también con el teclado.

```
<button id="boton-favorito" type="button">Marcar como favorito</button>
```

```
.favorito {
  background-color: gold;
}
```

```
const boton = document.querySelector("#boton-favorito");

if (boton !== null) {
  boton.addEventListener("click", () => {
    // toggle() añade la clase si falta y la quita si ya estaba.
    boton.classList.toggle("favorito");
  });
}
```

Ejemplo 4. Responder a una tecla

El evento `keydown` permite reaccionar a una tecla concreta. Comprueba `event.key` en vez del código numérico de la tecla:

```
<p id="aviso-tecla">Pulsa Escape para cerrar este aviso.</p>
```

```
const aviso = document.querySelector("#aviso-tecla");

if (aviso !== null) {
  document.addEventListener("keydown", evento => {
    if (evento.key === "Escape") {
      aviso.hidden = true;
```

```
    }
  });
}
```

Ejemplo 5. Cambiar el color de fondo al elegir una opción

El evento `change` se produce cuando se confirma una nueva opción de una lista desplegable.

```
<label for="color-fondo">Elige un color de fondo:</label>
<select id="color-fondo">
  <option value="white">Blanco</option>
  <option value="lightblue">Azul claro</option>
  <option value="lightgreen">Verde claro</option>
</select>
```

```
const selectorColor = document.querySelector("#color-fondo");

if (selectorColor !== null) {
  selectorColor.addEventListener("change", () => {
    document.body.style.backgroundColor = selectorColor.value;
  });
}
```